RP2350-LCD-1.47-A

From Waveshare Wiki
Jump to: navigation, search

Overview

Introduction

RP2350-LCD-1.47-A is a low-cost, high-performance MCU board designed by Waveshare. Despite its compact size, the board features a 1.47inch LCD screen, TF card slot, RGB LEDs and other peripherals, making it easy to develop and embed into products.

RP2350-LCD-1.47-A



(https://www.waveshare.com/RP2350-LCD-1.47-A.htm)

RP2350, USB Type-C

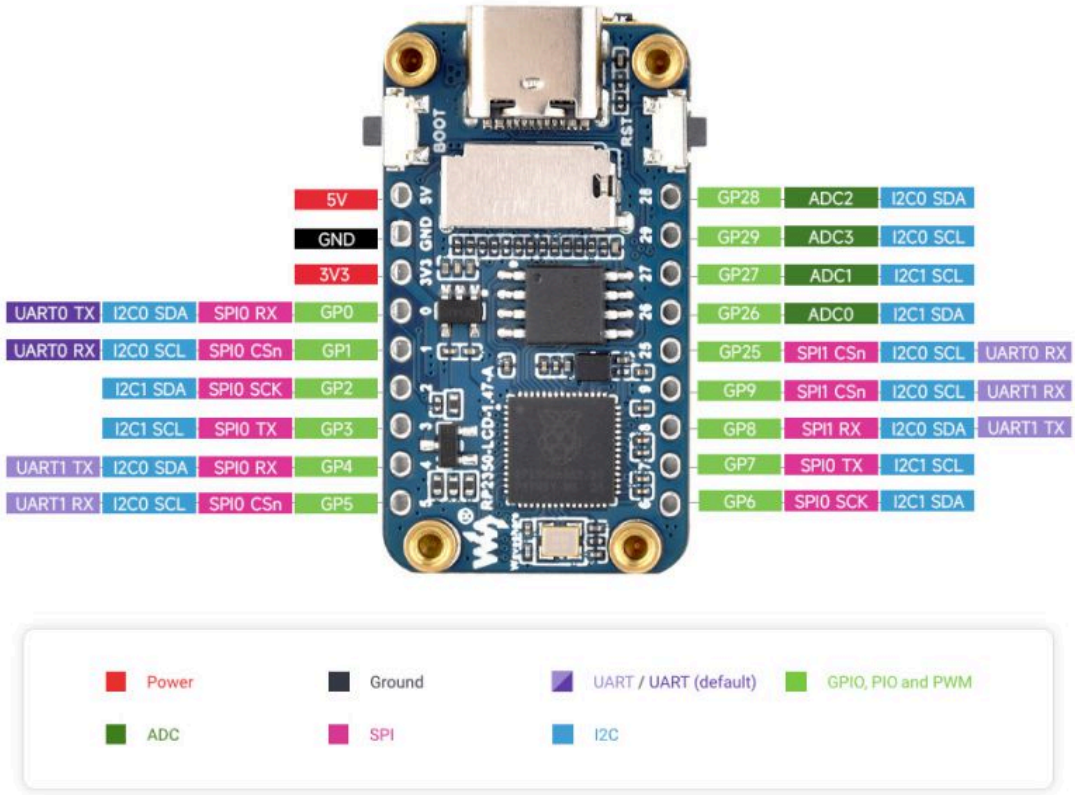
Features

- RP2350 microcontroller chip designed by Raspberry Pi
- Adopts unique dual-core and dual-architecture design, equipped with dual-core ARM Cortex-M33 processor and dual-core Hazard3 RISC-V core processor, flexible clock running up to 150 MHz
- Built-in 520KB of SRAM and 16MB of on-chip Flash
- Type-C port, keeps it up-to-date and easy to use
- Onboard 1.47inch LCD display, 172×320 resolution, 262K color
- USB1.1 host and slave device support
- Low-power sleep and dormant modes
- Drag-and-drop programming via USB mass storage
- Temperature sensor
- Accurate clock and timer on-chip
- On-chip accelerated floating-point library

Specifications

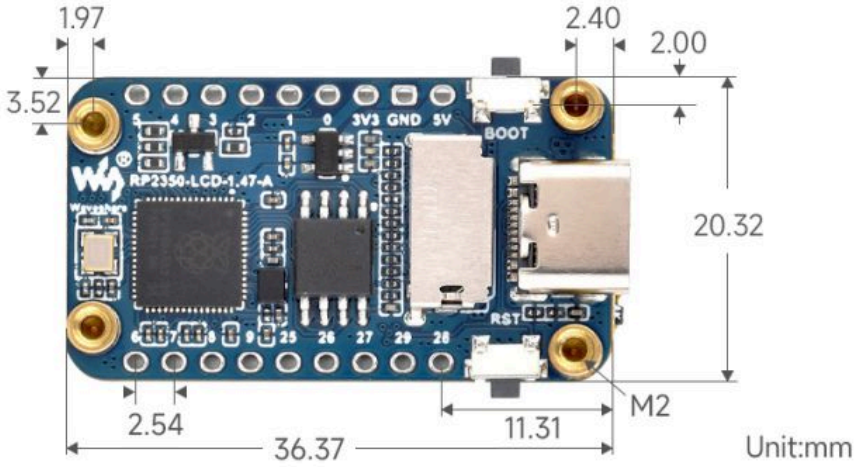
LCD parameters			
Display Chip	ST7789V3	Display Interface	SPI
Resolution	172(H)RGB x 320(V)	Display Size	17.39mm × 32.35 mm
Display Panel	IPS	Pixel Pitch	0.0337mm × 0.1011 mm

Pinout Definition



(/wiki/File:RP2350-LCD-1.47-A-details-inter.jpg)

Dimensions



(/wiki/File:RP2350-LCD-1.47-A-details-size.jpg)

Pico Getting Started

Firmware Download

- MicroPython Firmware Download [Expand]

- C_Blink Firmware Download [Expand]

Basic Introduction

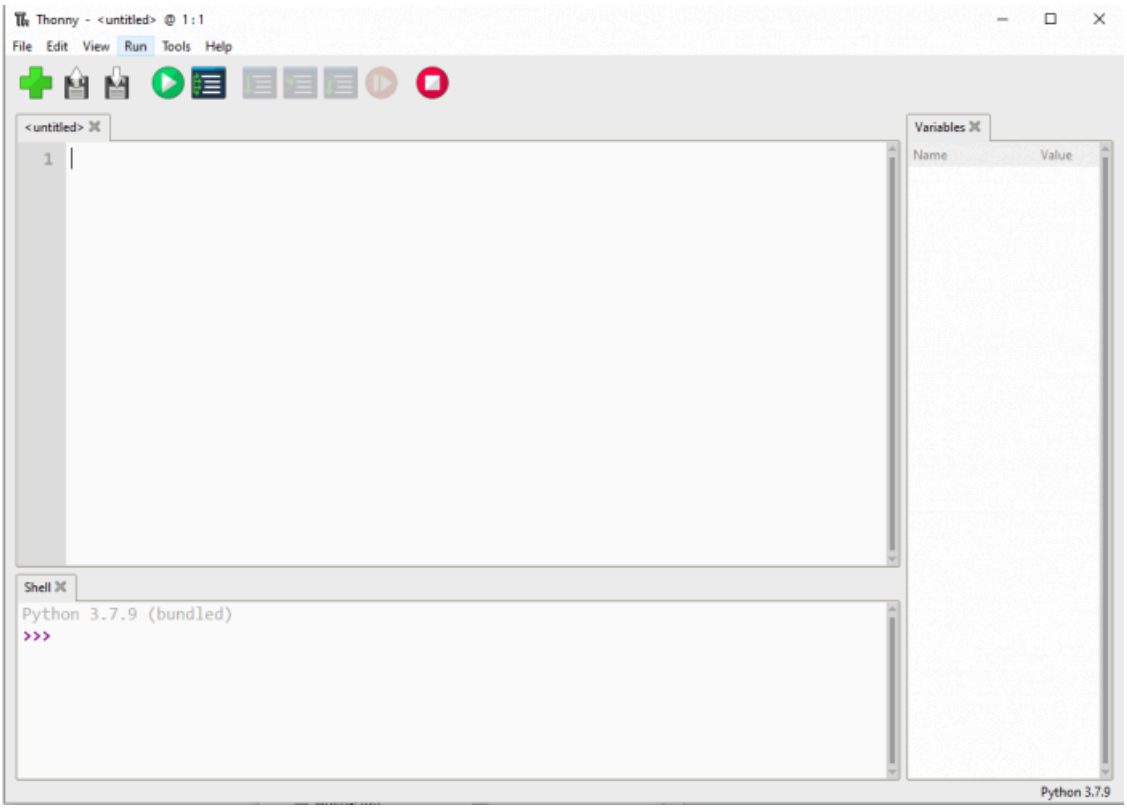
Raspberry Pi Pico Basics (https://www.waveshare.com/wiki/Pico_Basic_Introduction)

MicroPython Series

Install Thonny IDE

To facilitate the development of Pico/Pico2 boards with MicroPython on a computer, it is recommended to download the Thonny IDE

- Download Thonny IDE and follow the steps to install, the installation packages are all Windows versions, please refer to Thonny's official website for other versions
  - Thonny IDE official download link (https://github.com/thonny/thonny/releases/download/v3.3.3/thonny-3.3.3.exe)
  - Thonny IDE download link (https://files.waveshare.com/wiki/common/Thonny-3.3.3.zip)
  - Thonny official website (https://thonny.org/)
- After installation, configure the language and motherboard environment for the first use. Since we are using Pico/Pico2, pay attention to selecting the Raspberry Pi option for the motherboard environment

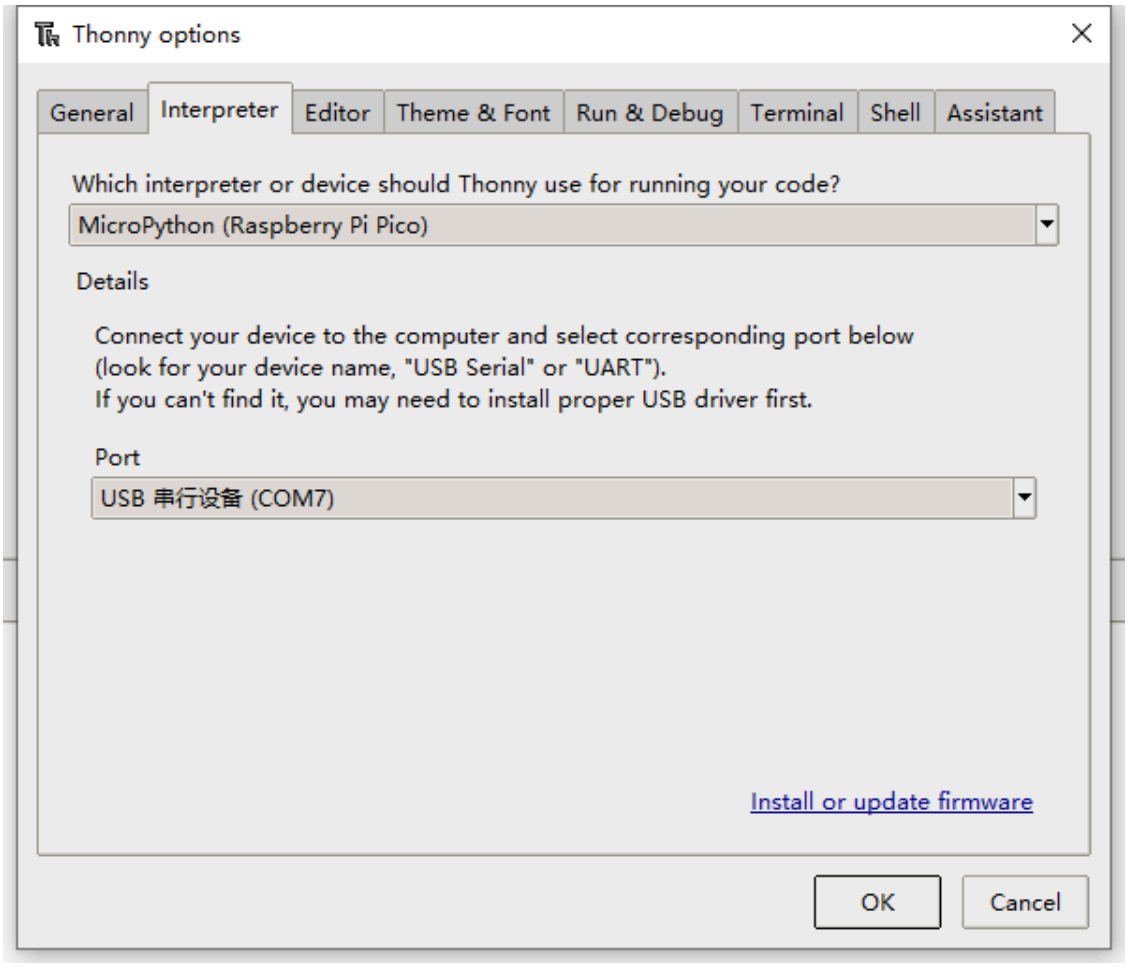


(/wiki/File:Pico-R3-Tonny1.png)

- Configure MicroPython environment and choose Pico/Pico2 port
  - Connect Pico/Pico2 to your computer first, and in the lower right corner of Thonny left-click on the configuration environment option --> select Configure interpreter
  - In the pop-up window, select MicroPython (Raspberry Pi Pico), and choose the corresponding port



(/wiki/File:1050px-Raspberry-Pi-Pico-Basic-Kit-M-2.png)



(/wiki/File:Raspberry-Pi-Pico-Basic-Kit-M-3.png)

Flash Firmware

- Click OK to return to the Thonny main interface, download the corresponding firmware library and flash it

to the device, and then click the Stop button to display the current environment in the Shell window

- **Note: For the Pico series board, you can directly use the firmware provided by MicroPython official. For the RP series board, please use the firmware provided below or in the program package.**
  - Pico / Pico2 Official firmware library (https://micropython.org/download/?vendor=Raspberry%20Pi)
  - RP2040 firmware library (https://files.waveshare.com/wiki/RP2350-Plus/WAVESHARE-RP2040-Board.zip)
  - RP2350A firmware library (https://files.waveshare.com/wiki/RP2350-Plus/WAVESHARE-RP2350A-Board.zip)
  - RP2350B firmware library (https://files.waveshare.com/wiki/RP2350-Plus/WAVESHARE-RP2350B-Board.zip)
- Steps to compile the latest firmware
  - https://github.com/micropython/micropython/tree/master/ports/rp2 (https://github.com/micropython/micropython/tree/master/ports/rp2)
- How to download the firmware library for Pico/Pico2 in windows: After holding down the BOOT button and connecting to the computer, release the BOOT button, a removable disk will appear on the computer, copy the firmware library into it
- How to download the firmware library for RP2040/RP2350 in windows: After connecting to the computer, press the BOOT key and the RESET key at the same time, release the RESET key first and then release the BOOT key, a removable disk will appear on the computer, copy the firmware library into it (you can also use the Pico/Pico2 method)



(/wiki/File:Raspberry-Pi-Pico2-Python.png)

MicroPython Series Tutorials

【MicroPython】machine.Pin class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91_Machine.Pin_Functions)

【MicroPython】machine.PWM class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.PWM_Function)

【MicroPython】machine.ADC class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.ADC_Function)

【MicroPython】machine.UART class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.UART_Function)

【MicroPython】machine.I2C class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.I2C_Function)

【MicroPython】machine.SPI class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.SPI_Function)

【MicroPython】rp2.StateMachine class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91PIO_Function)

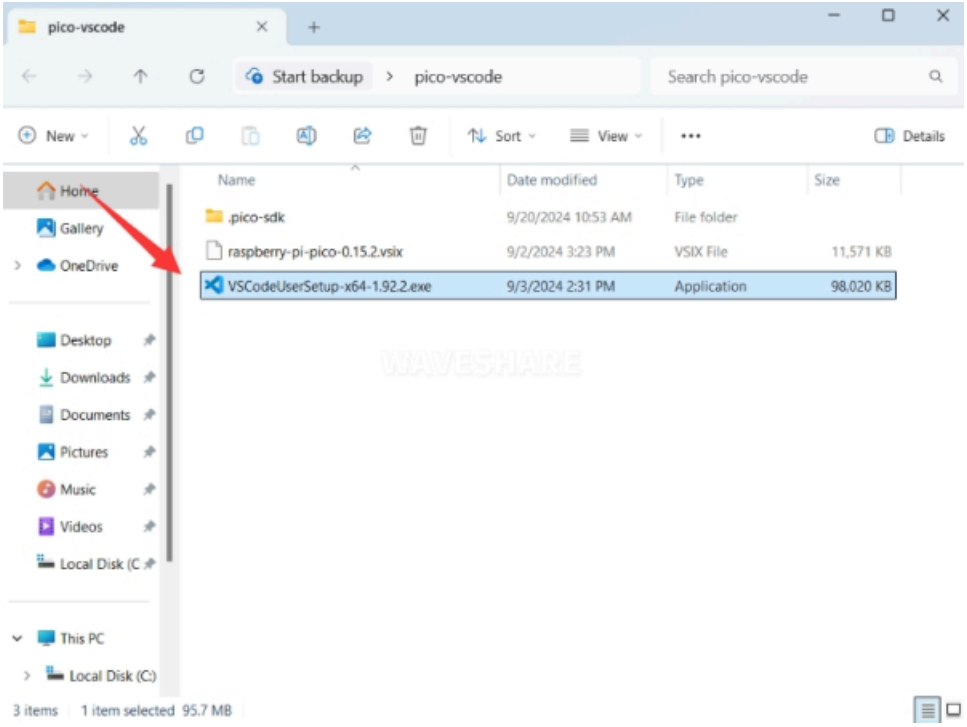
C/C++ Series

For C/C++, it is recommended to use Pico VSCode for development. This is a Microsoft Visual Studio Code extension designed to make it easier for you to create, develop, and debug projects for the Raspberry Pi Pico series development boards. No matter if you are a beginner or an experienced professional, this tool can assist you in developing Pico with confidence and ease. Here's how to install and use the extension.

- Official website tutorial: https://www.raspberrypi.com/news/pico-vscode-extension/ (https://www.raspberrypi.com/news/pico-vscode-extension/)
- This tutorial is suitable for Raspberry Pi Pico, Pico2 and the RP2040 and RP2350 series development boards developed by Waveshare
- The development environment defaults to Windows11. For other environments, please refer to the official tutorial for installation

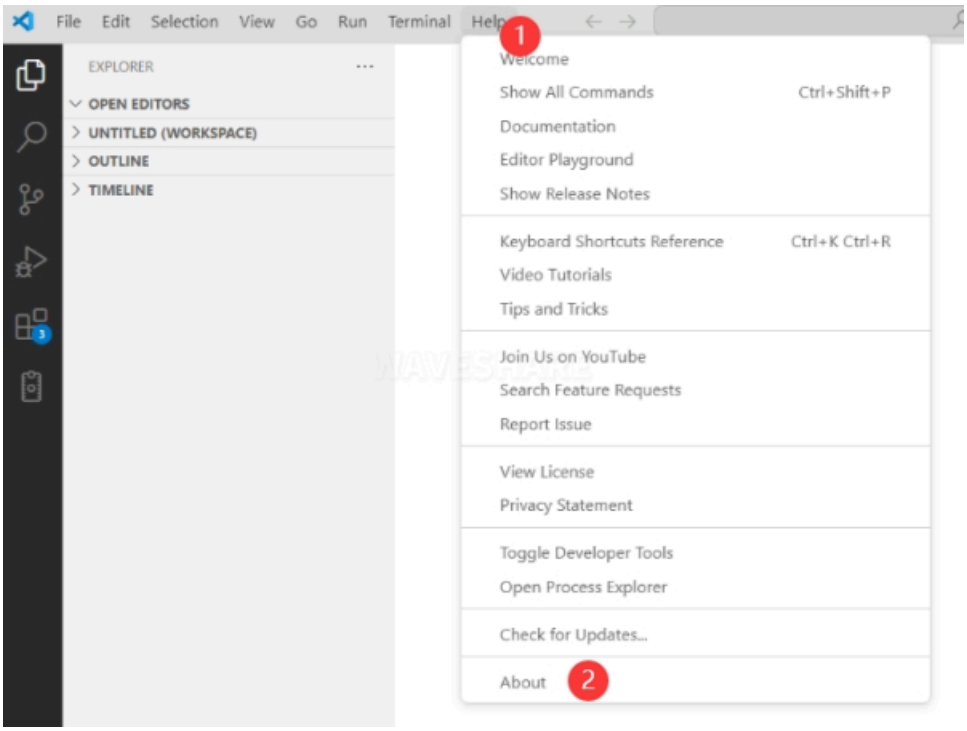
Install VSCode

1. First, click to download pico-vscode package (https://drive.google.com/file/d/18-KDNrQlI0KuTMdS6W5iblUGaGm3FbVJ/view?usp=sharing), unzip and open the package, double-click to install VSCode

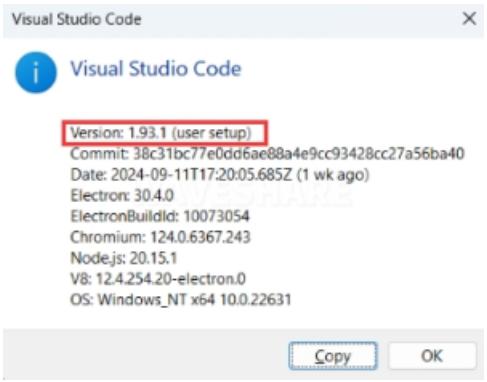


(/wiki/File:Pico-vscode-1.png)

Note: If vscode is installed, check if the version is v1.87.0 or later



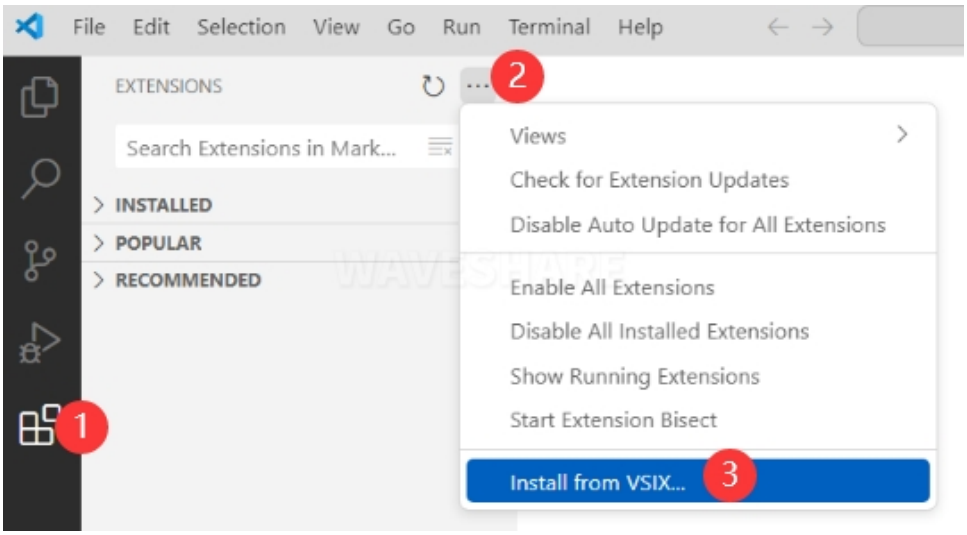
(/wiki/File:Pico-vscode-2.png)



(/wiki/File:Pico-vscode-3.png)

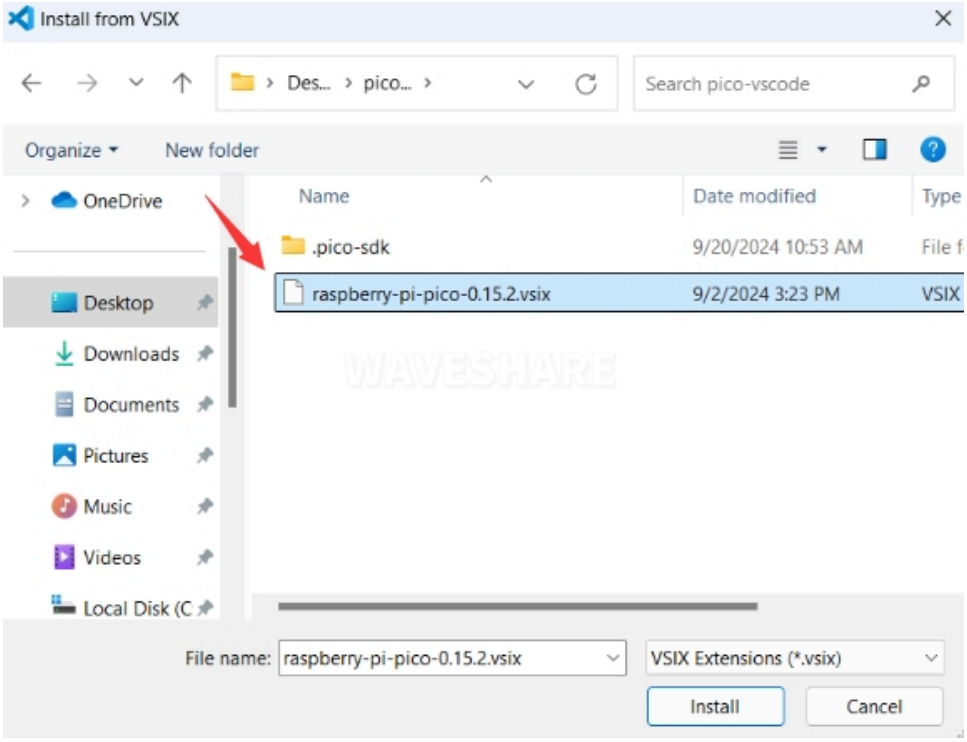
Install Extension

1. Click Extensions and select Install from VSIX



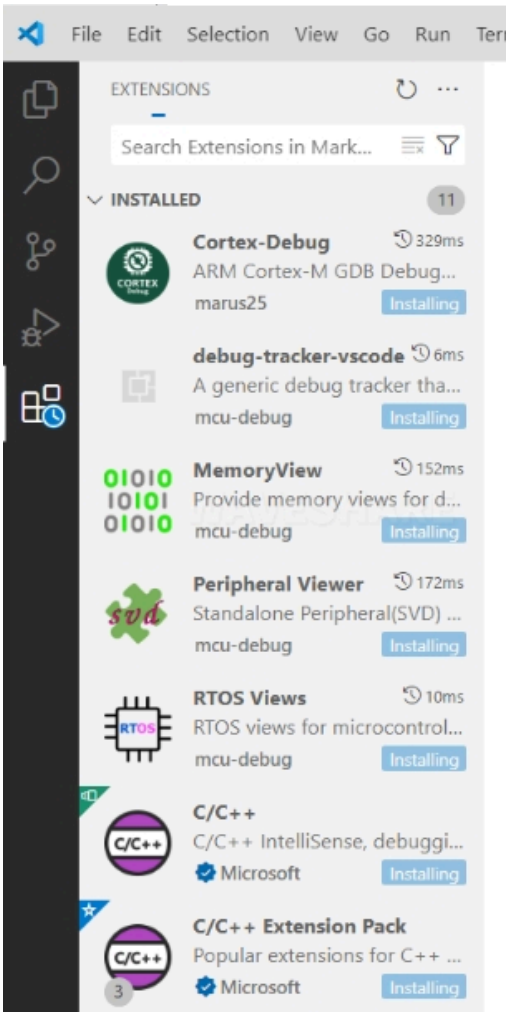
(/wiki/File:Pico-vscode-4.png)

2. Select the package with the vsix suffix and click Install



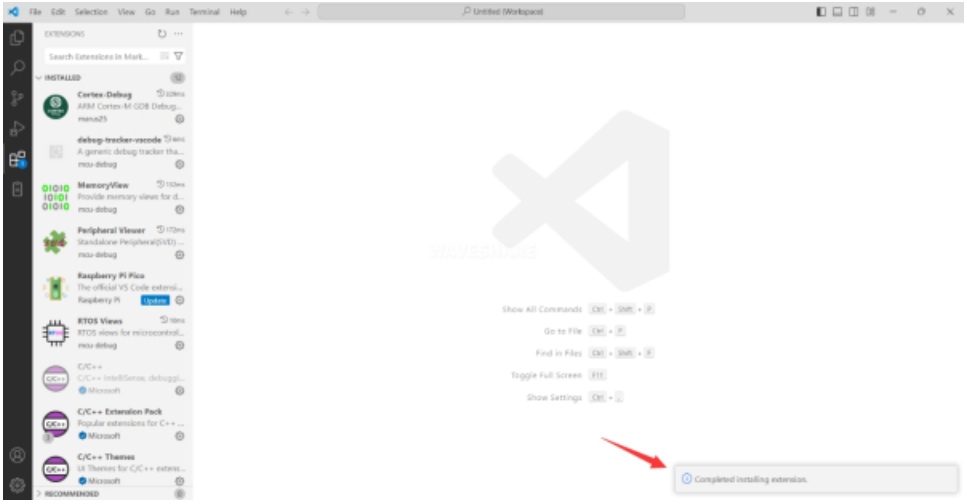
(/wiki/File:Pico-vscode-5.png)

3. Then vscode will automatically install raspberry-pi-pico and its dependency extensions, you can click Refresh to check the installation progress



(/wiki/File:Pico-vscode-6.png)

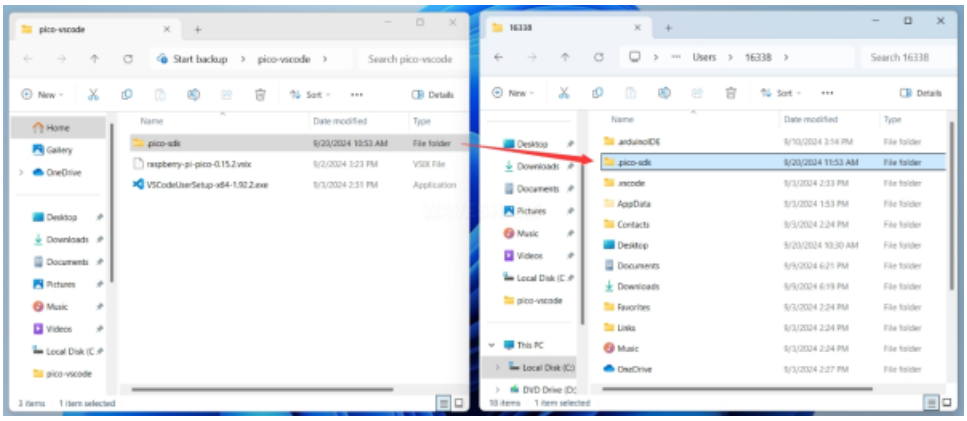
4. The text in the right lower corner shows that the installation is complete. Close VSCode



(/wiki/File:Pico-vscode-7.png)

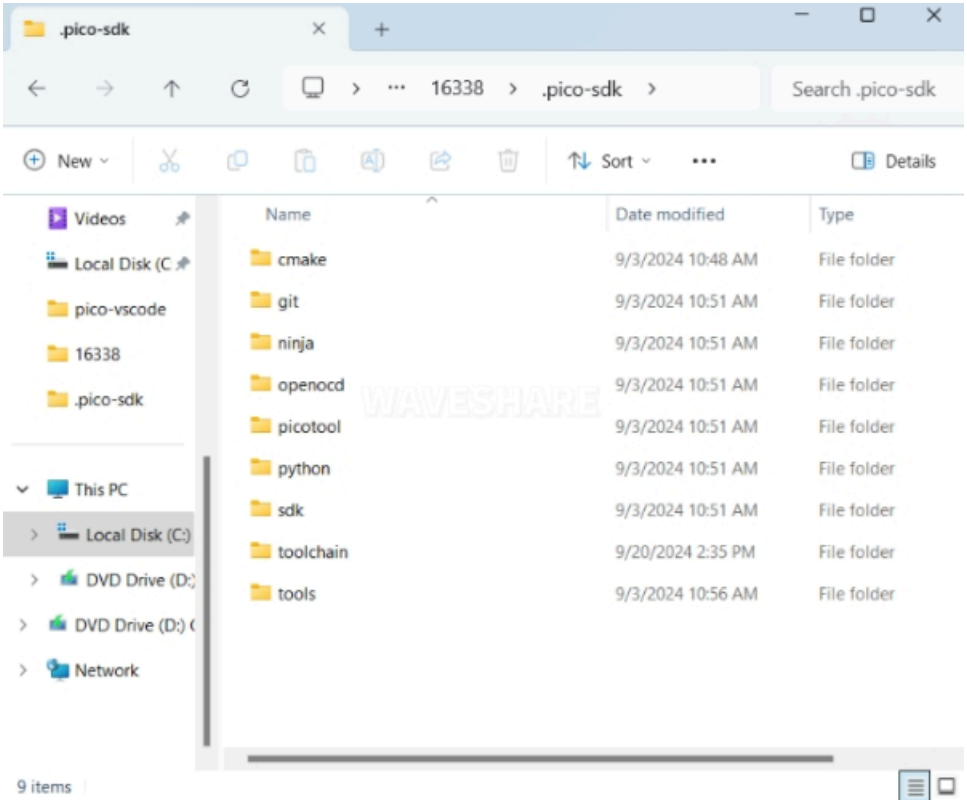
Configure Extension

1. Open directory C:\Users\username and copy the entire .pico-sdk to that directory



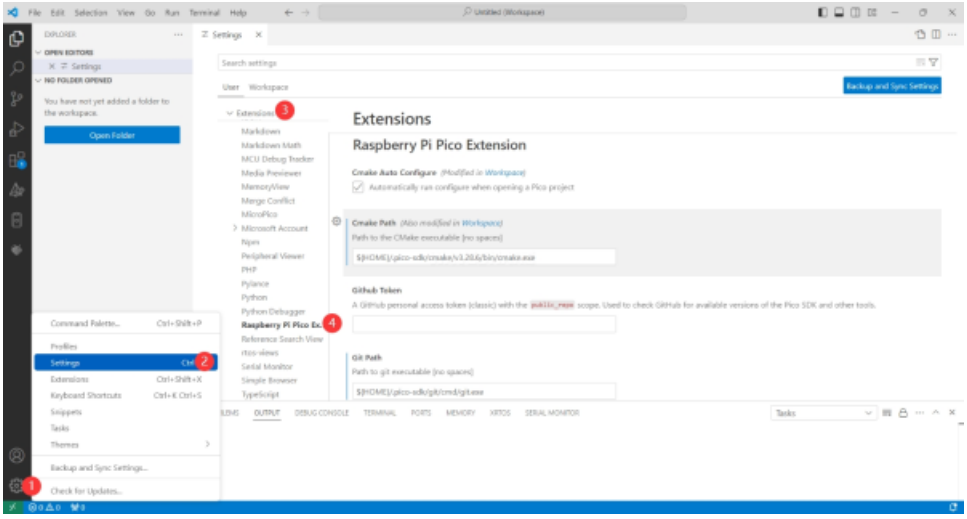
(/wiki/File:Pico-vscode-8.png)

2. The copy is completed



(/wiki/File:Pico-vscode-9.png)

3. Open vscode and configure the paths for the Raspberry Pi Pico extensions



(/wiki/File:Pico-vscode-10.png)

The configuration is as follows:

```
Cmake Path:
${HOME}/.pico-sdk/cmake/v3.28.6/bin/cmake.exe

Git Path:
${HOME}/.pico-sdk/git/cmd/git.exe

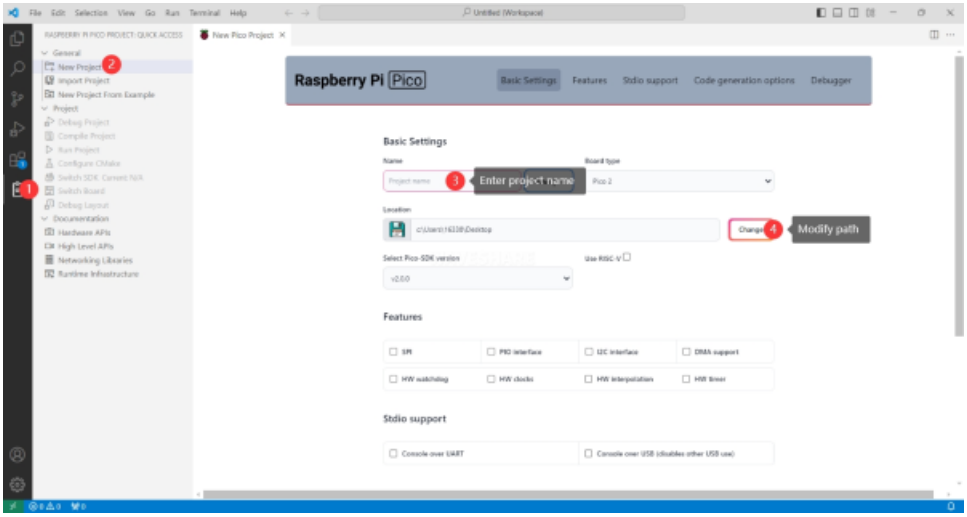
Ninja Path:
${HOME}/.pico-sdk/ninja/v1.12.1/ninja.exe

Python3 Path:
${HOME}/.pico-sdk/python/3.12.1/python.exe
```

New Project

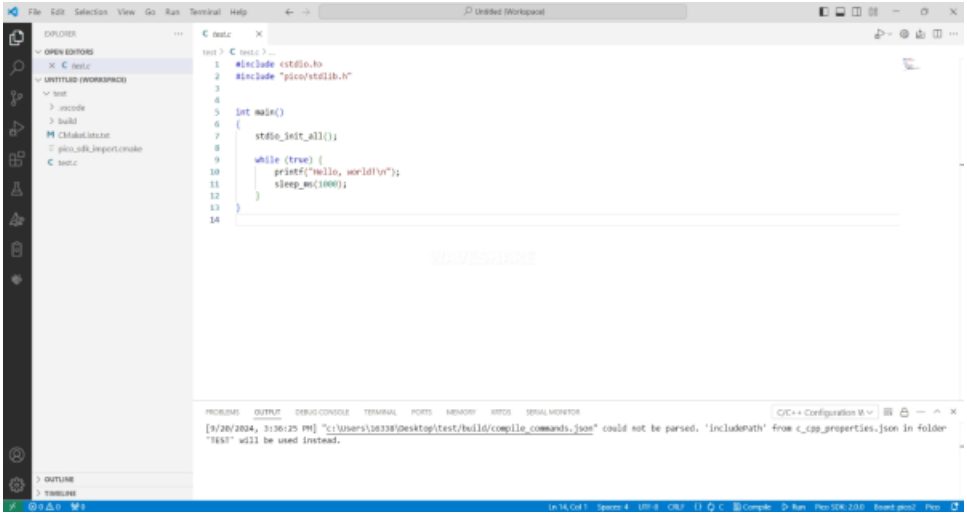
1. The configuration is complete, create a new project, enter the project name, select the path, and click Create to create the project

   To test the official example, you can click on the Example next to the project name to select



(/wiki/File:Pico-vscode-11.png)

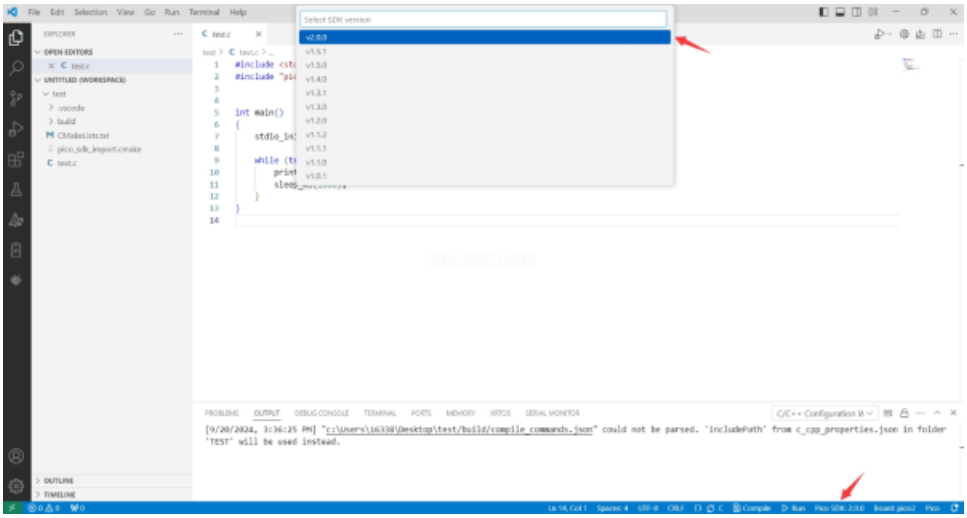
2. The project is created successfully



(/wiki/File:Pico-vscode-12.png)

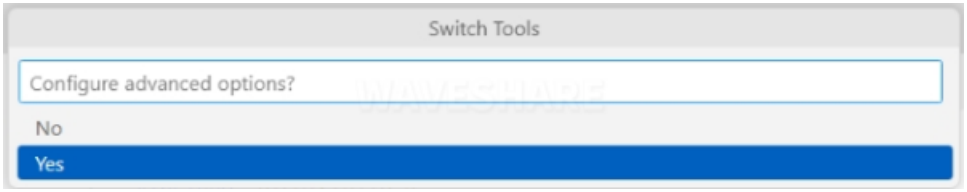
Compile Project

1. Select the SDK version



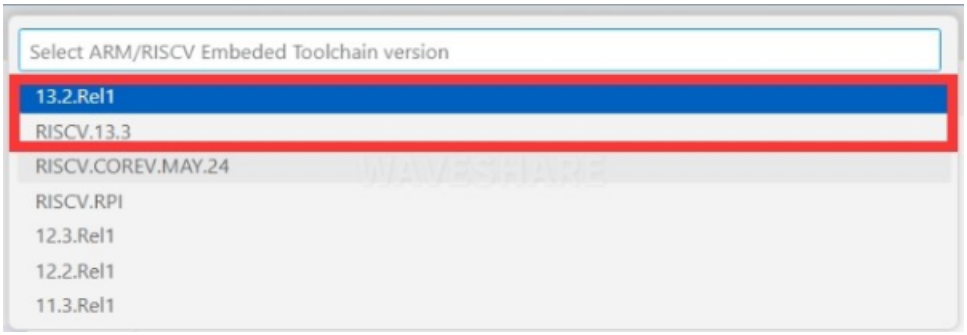
(/wiki/File:Pico-vscode-13.png)

2. Select Yes for advanced configuration



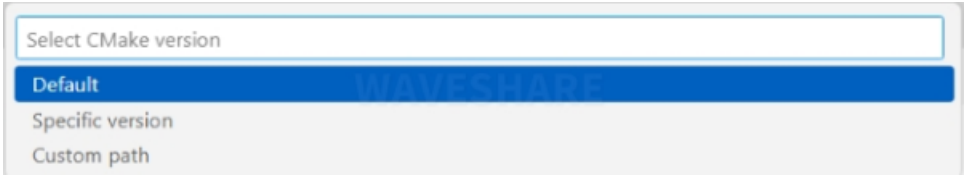
(/wiki/File:Pico-vscode-14.png)

3. Choose the toolchain, 13.2.Rel1 is applicable for ARM cores, RISCV.13.3 is applicable for RISCV cores. You can select either based on your requirements



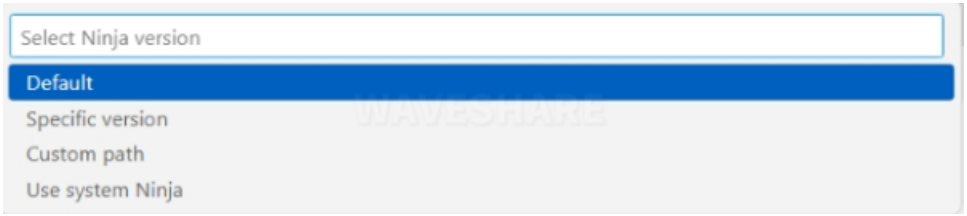
(/wiki/File:Pico-vscode-15.png)

4. Select Default for CMake version (the path configured earlier)



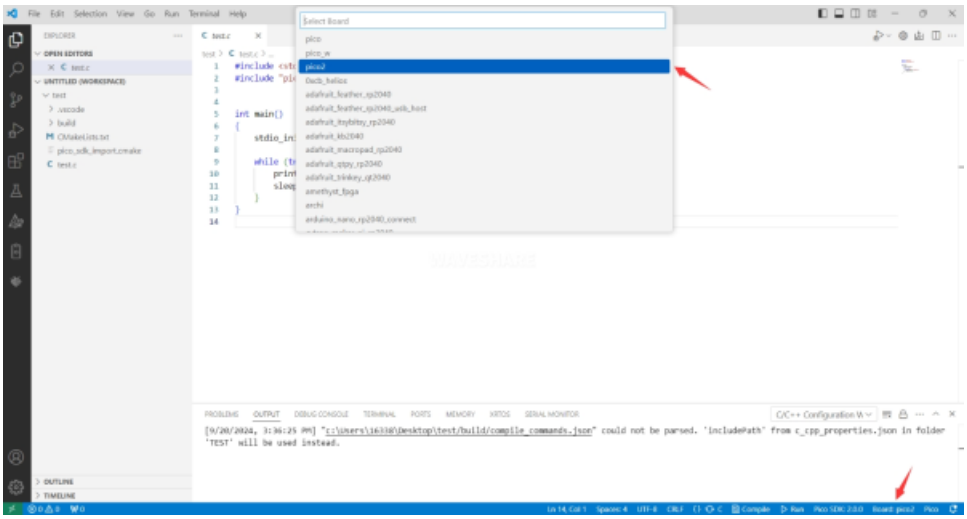
(/wiki/File:Pico-vscode-16.png)

5. Select Default for Ninja version



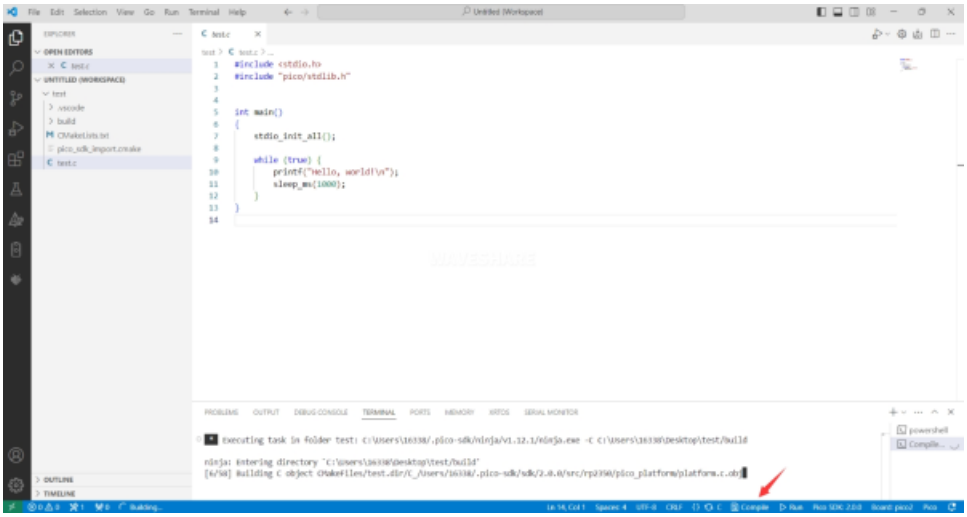
(/wiki/File:Pico-vscode-17.png)

6. Select the development board



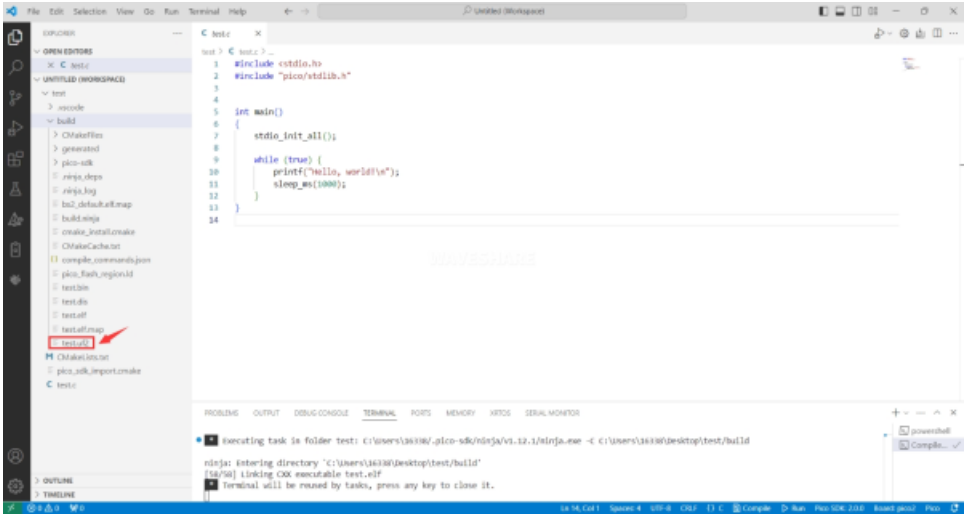
(/wiki/File:Pico-vscode-18.png)

7. Click Compile to compile



(/wiki/File:Pico-vscode-19.png)

8. The .uf2 format file is successfully compiled



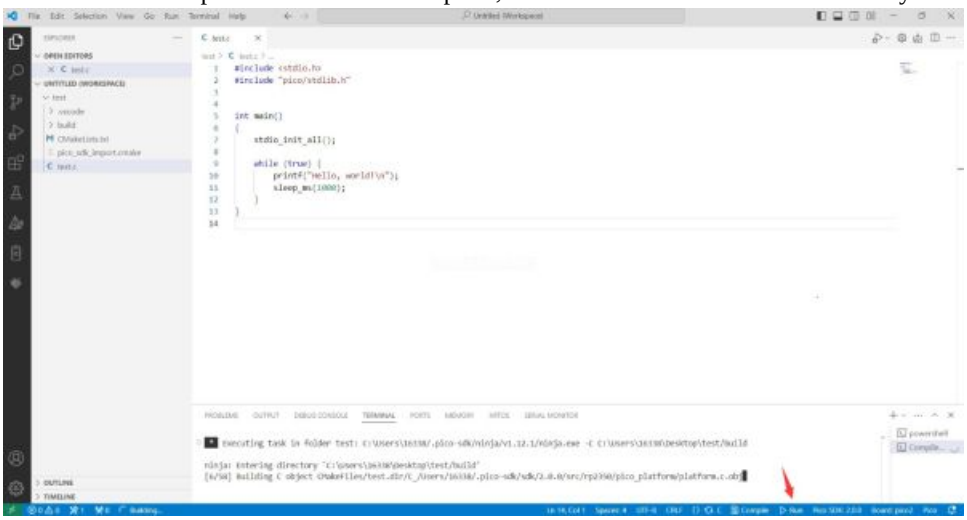
(/wiki/File:Pico-vscode-20.png)

Flash Firmware

Here are two methods for flashing firmware

1. Flash firmware using the pico-vscode plugin

   Connect the development board to the computer, click Run to flash the firmware directly

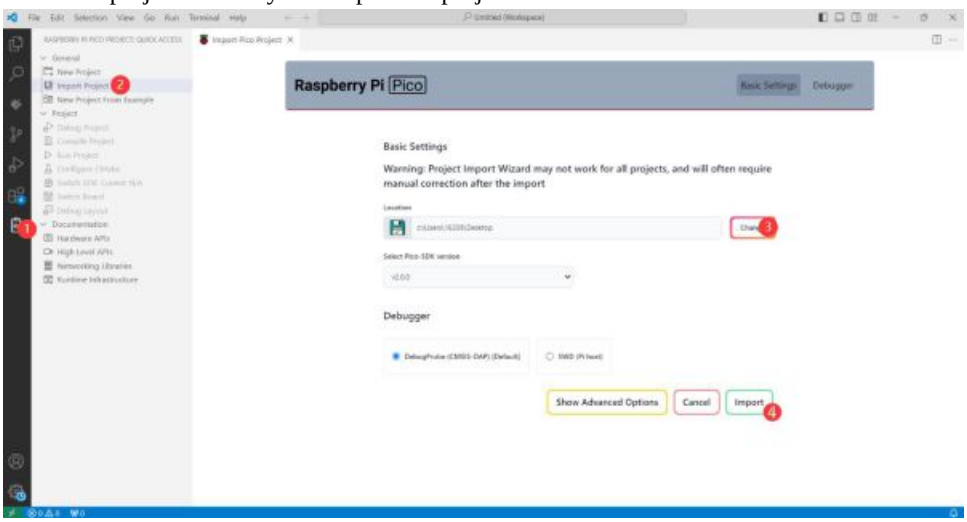


   (/wiki/File:Pico-vscode-24.jpg)
2. Flash the firmware manually

```
1. Press and hold the Boot button
2. Connect the development board to the computer
3. Then the computer will recognize the development board as a USB device.
4. Copy the .uf2 file to the USB drive, and the device will automatically re
start, indicating successful program flashing.
```

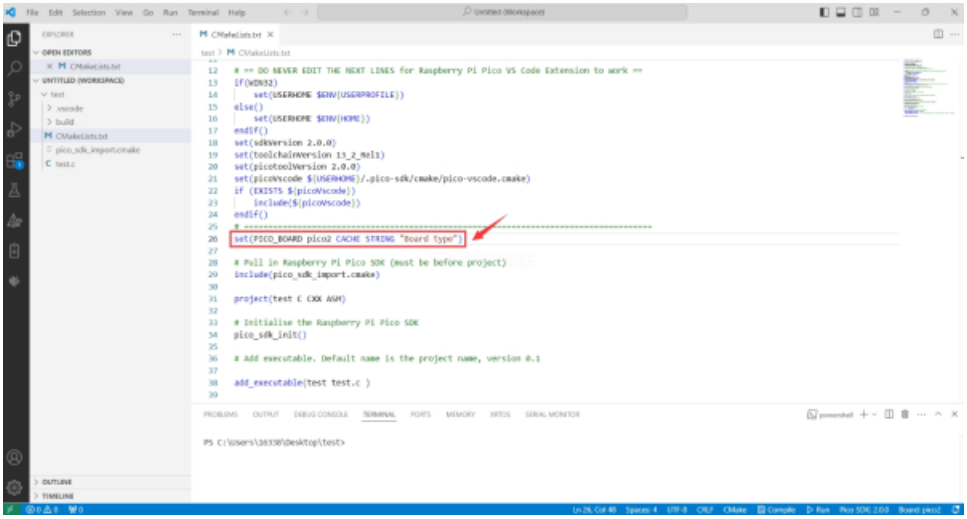
Import Project

1. Select the project directory and import the project



(/wiki/File:Pico-vscode-23.jpg)

2. The Cmake file of the imported project cannot have Chinese (including comments), otherwise the import may fail
3. To import your own project, you need to add a line of code to the Cmake file to switch between pico and pico2 normally, otherwise even if pico2 is selected, the compiled firmware will still be suitable for pico

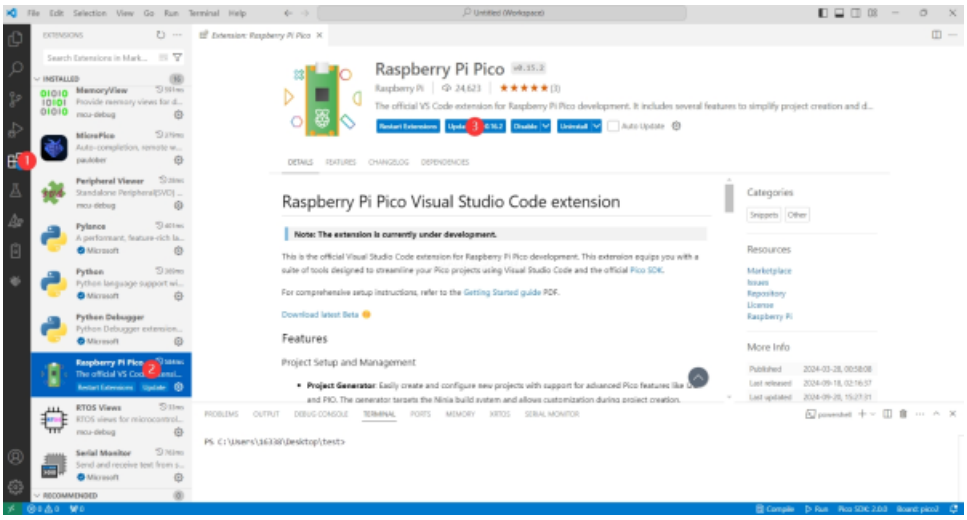


(/wiki/File:Pico-vscode-21.png)

```
set(PICO_BOARD pico CACHE STRING "Board type")
```

Update Extension

1. The extension version in the offline package is 0.15.2, and you can also choose to update to the latest version after the installation is complete

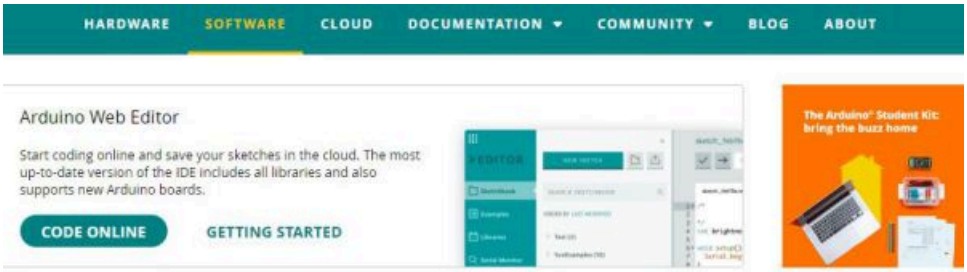


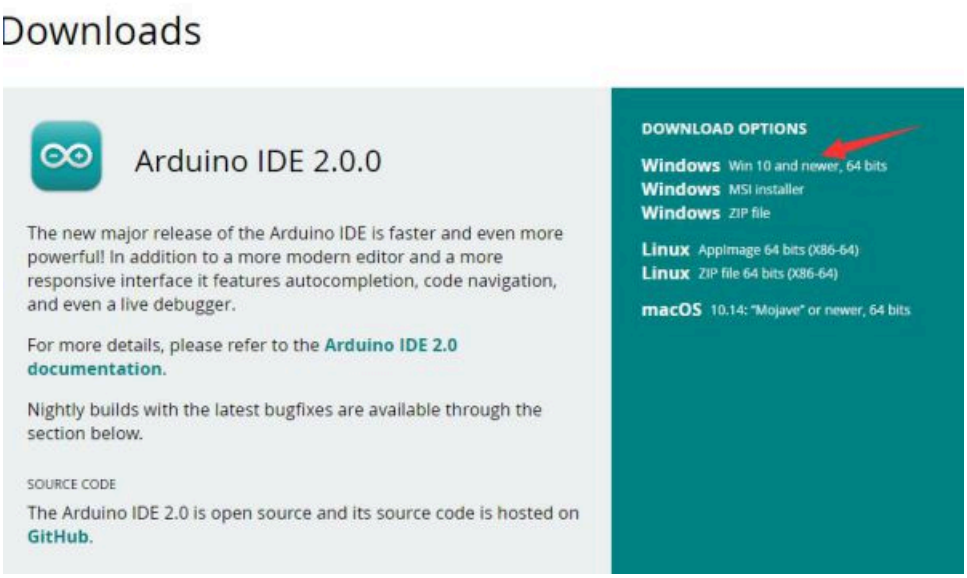
(/wiki/File:Pico-vscode-22.png)

Arduino IDE Series

Install Arduino IDE

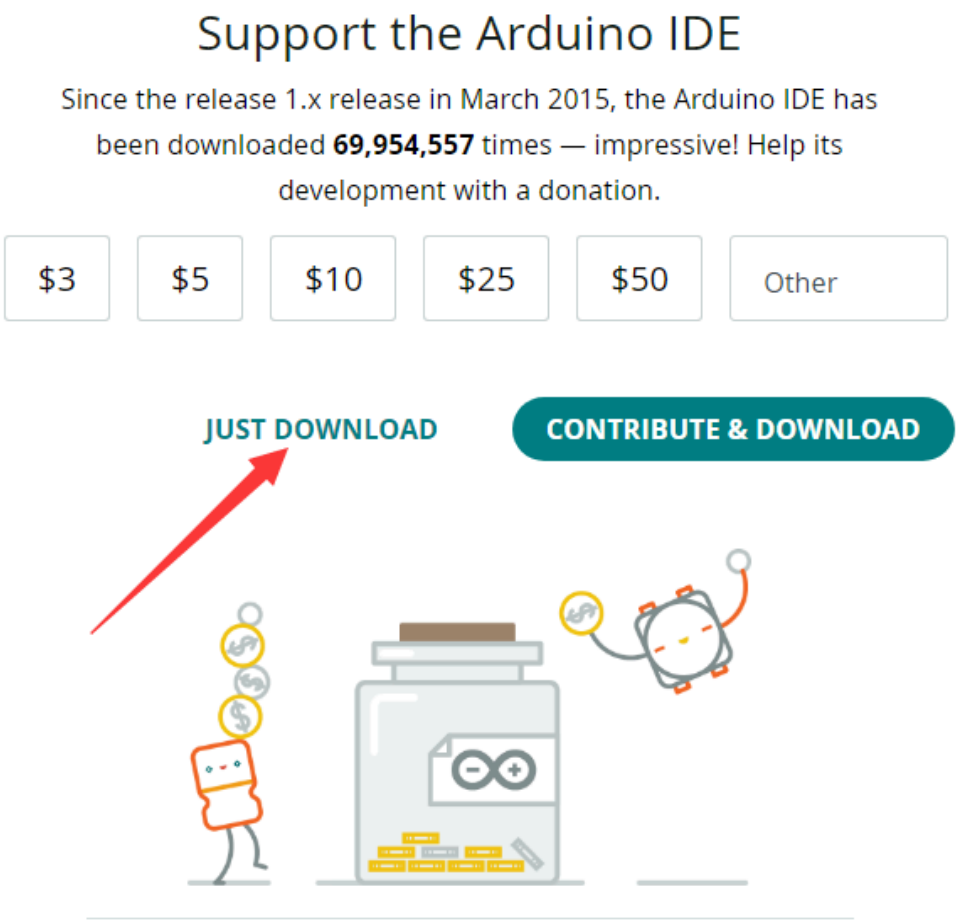
1. First, go to Arduino official website (https://www.arduino.cc/) to download the installation package of the Arduino IDE.





(/wiki/File:Arduino%E4%B8%8B%E8%BD%BD2.0%E7%89%88%E6%9C%AC.jpg)

2. Here, you can select Just Download.



(/wiki/File:%E4%BB%85%E4%B8%8B%E8%BD%BD%E4%B8%8D%E6%8D%90%E8%B5%A0.png)

3. Once the download is complete, click Install.



(/wiki/File:IDE%E5%AE%89%E8%A3%85%E6%B0%B4%E5%8D%B0-1.gif)

Notice: During the installation process, it will prompt you to install the driver, just click Install

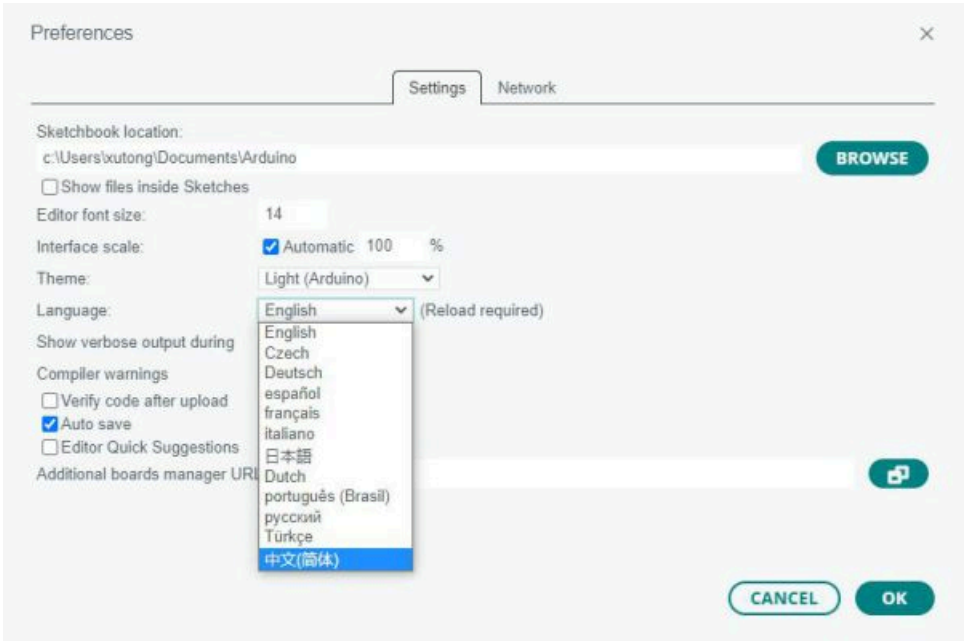
Arduino IDE Interface

1. After the first installation, when you open the Arduino IDE, it will be in English. You can switch to other languages in File --> Preferences, or continue using the English interface.



(/wiki/File:%E9%A6%96%E9%80%89%E9%A1%B9-%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87.jpg)

2. In the Language field, select the language you want to switch to, and click OK.



(/wiki/File:%E9%A6%96%E9%80%89%E9%A1%B9-%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87ok.jpg)

Install Arduino-Pico Core in Arduino IDE

1. Open the Arduino IDE, click on the file in the top left corner, and select Preferences

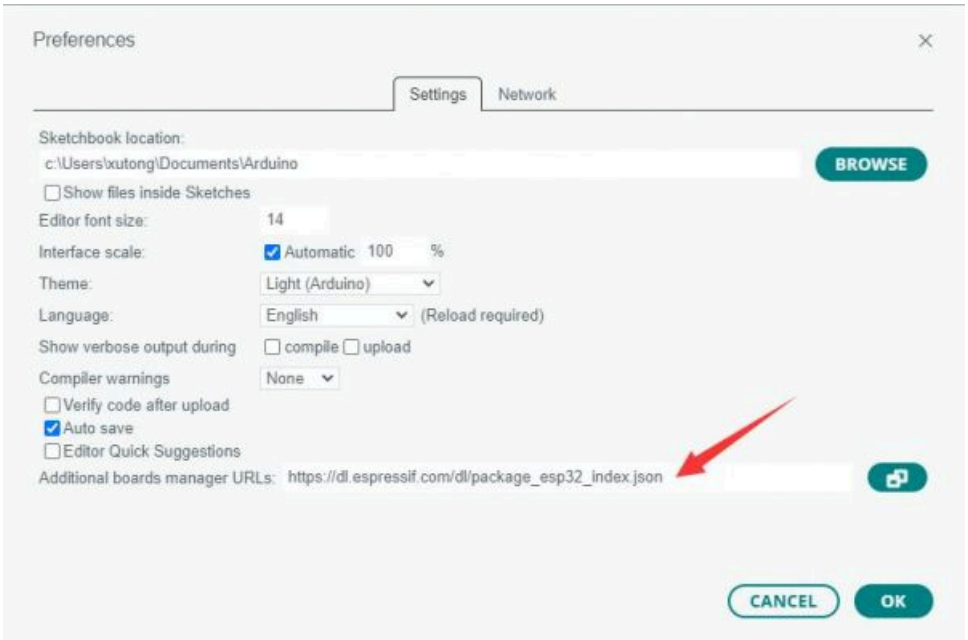


(/wiki/File:RoArm-M1_Tutorial04.jpg)

2. Add the following link to the attached board manager URL, and then click OK

   This link already includes board versions such as RP2040 and RP2350. Please visit arduino-pico (https://github.com/earlephilhower/arduino-pico) for the latest version files

```
https://github.com/earlephilhower/arduino-pico/releases/download/4.5.2/package_rp2040_index.json
```



(/wiki/File:RoArm-M1_Tutorial_II05.jpg)

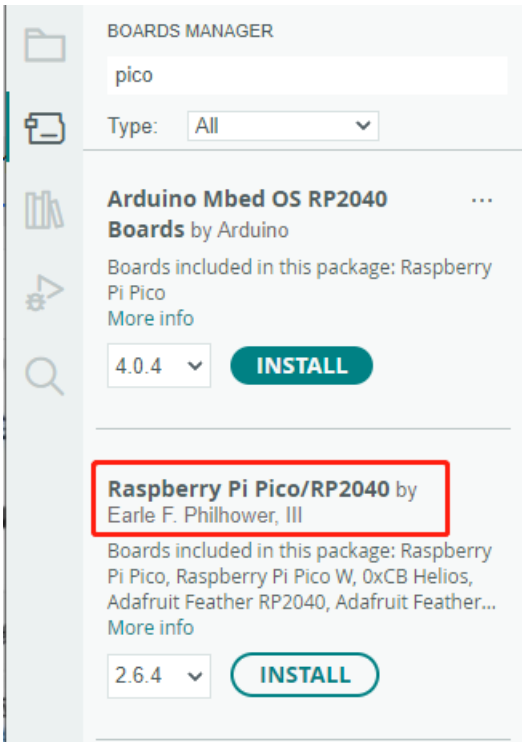
Note: If you already have an ESP32 board URL, you can use a comma to separate the URLs as follows:

```
https://dl.espressif.com/dl/package_esp32_index.json,https://github.com/earlephilhower/arduino-pico/releases/download/4.5.2/package_rp2040_index.json
```

3. Click Tools > Development Board > Board Manager > Search pico, as my computer has already been installed, it shows that it is installed



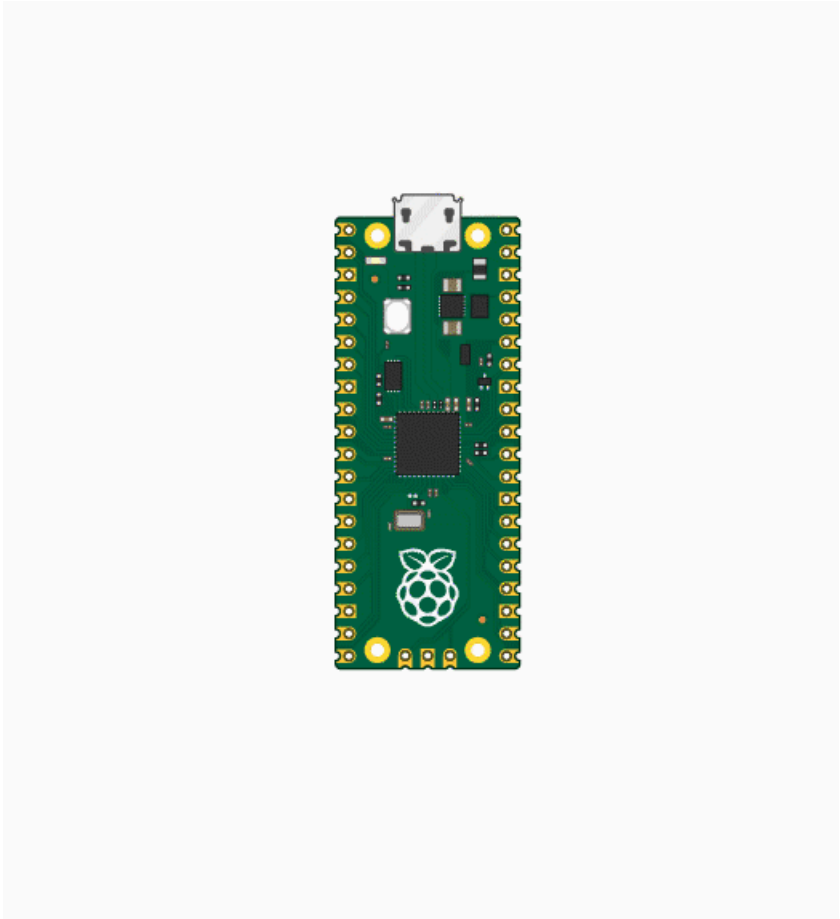
(/wiki/File:Pico_Get_Start_05.png)



(/wiki/File:Pico_Get_Start_06.png)

Upload Demo at the First Time

1. Press and hold the BOOTSET button on the Pico board, connect the pico to the USB port of the computer via the Micro USB cable, and release the button after the computer recognizes a removable hard disk (RPI-RP2).



(/wiki/File:Pico%E8%BF%9E%E6%8E%A5%E6%95%B0%E6%8D%AE%E7%BA%BF.gif)

2. Download the program and open D1-LED.ino under the arduino\PWM\D1-LED path
3. Click Tools --> Port, remember the existing COM, do not click this COM (the COM displayed is different on different computers, remember the COM on your own computer)



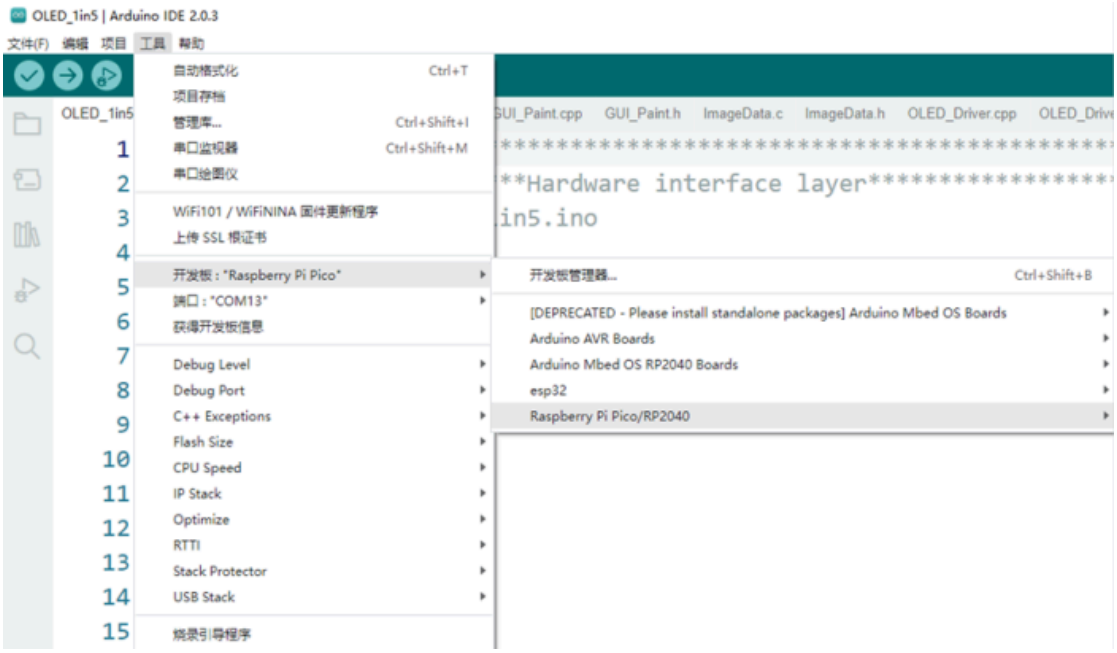
(/wiki/File:Pico%E8%BF%9E%E6%8E%A5%E5%89%8D%E7%AB%AF%E5%8F%A3.png)

4. Connect the driver board to the computer using a USB cable. Then, go to Tools > Port. For the first connection, select uf2 Board. After uploading, when you connect again, an additional COM port will appear

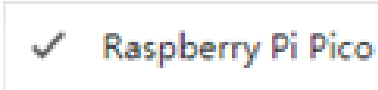


(/wiki/File:Pico%E8%BF%9E%E6%8E%A5%E5%90%8Euf2.png)

5. Click Tools > Development Board > Raspberry Pi Pico > Corresponding models (Raspberry Pi Pico, Raspberry Pi Pico 2, etc.)



(/wiki/File:%E5%B7%A5%E5%85%B7pico%E5%BC%80%E5%8F%91%E6%9D%BF.png)



(/wiki/File:Arduono-Raspberrypi_pico.png)

6. After setting it up, click the right arrow to upload the program



(/wiki/File:Pico%E4%B8%8A%E4%BC%A0%E7%A8%8B%E5%BA%8F.png)

- **If issues arise during this period, and if you need to reinstall or update the Arduino IDE version, it is necessary to uninstall the Arduino IDE completely. After uninstalling the software, you need to manually delete all contents within the C:\Users\[name]\AppData\Local\Arduino15 folder (you need to show hidden files to see this folder). Then, proceed with a fresh installation.**

Open Source Demos

MircoPython video demo (github) (https://github.com/waveshareteam/Pico_MircoPython_Examples)

MicroPython firmware/Blink demos (C) (https://files.waveshare.com/wiki/common/Raspberry_Pi_Pico_Demo.zip)

Raspberry Pi official C/C++ demo (github) (https://github.com/raspberrypi/pico-examples/)

Raspberry Pi official MicroPython demo (github) (https://github.com/raspberrypi/pico-micropython-examples)

Arduino official C/C++ demo (github) (https://github.com/earlephilhower/arduino-pico)

GUI API Details

If you have used our SPI screen before, you should be familiar with this example program

C

Underlying Hardware Interfaces

We have encapsulated the underlying layer, and due to different hardware platforms, the internal

implementation is different. If you need to understand the internal implementation, you can check the corresponding directory

You can see a lot of definitions in DEV_Config.c(.h), which can be found in the directory: c\lib\Config

- Data type:

```
#define UBYTE   uint8_t
#define UWORD   uint16_t
#define UDOUBLE uint32_t
```

- Module initialization and exit processing:

```
void DEV_Module_Init(void);
void DEV_Module_Exit(void);
Note:
1. Here is the processing of some GPIOs before and after using the LCD screen.
```

- GPIO read/write:

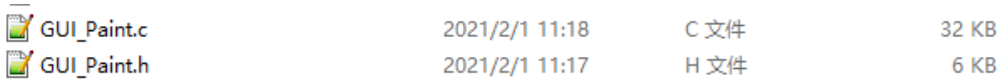
```
void    DEV_Digital_Write(UWORD Pin, UBYTE Value);
UBYTE   DEV_Digital_Read(UWORD Pin);
```

- SPI write data:

```
void DEV_SPI_WriteByte(UBYTE Value);
```

Upper Layer Applications

For the screen, what if you need to paint, display Chinese and English characters, display pictures, etc., these are all done by the upper layer applications. Many of you have asked about some graphics processing, and we have provided some basic functions in the directory The GUI can be found in the directory: c\lib\GUI\GUI_Paint.c(.h)



(/wiki/File:LCD_PICO_GUI_1.png)

In the following directory are the character fonts that the GUI depends on: c\lib\Fonts



(/wiki/File:LCD_rpi_Font.png)

- New Image Property: Create a new image property that includes the name, width, height, angle of rotation, and color of the image cache

```
void Paint_NewImage(UWORD *image, UWORD Width, UWORD Height, UWORD Rotate, UWORD Color)
Parameter:
        image:  The name of the image cache, which is actually a pointer to the first address of the image cache;
        Width:  The width of the image cache;
        Height: The height of image cache;
        Rotate: The angle of image rotation;
        Color:  The initial color of the image;
```

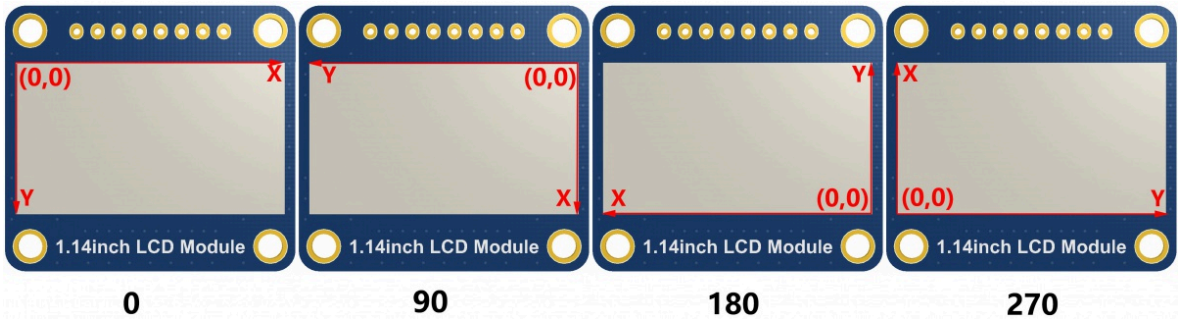
- Select Image Cache: The purpose of selecting the image cache is that you can create multiple image properties, and there can be multiple image caches. You can select each image you create

```
void Paint_SelectImage(UBYTE *image)
Parameter:
        image: The name of the image cache, which is actually a pointer to the first address of the image cache;
```

- Image Rotation: Set the rotation angle of the selected image, preferably after Paint_SelectImage (), you can choose to rotate 0, 90, 180, 270

```
void Paint_SetRotate(UWORD Rotate)
Parameter:
        Rotate: Image rotation angles can be chosen as ROTATE_0, ROTATE_90, ROTATE_180, ROTATE_270 corresponding to 0, 90, 180, 270 degrees
```

Note: Under different selection angles, the coordinates correspond to different starting pixels. Here taking 1.14 as an example, there are four images in sequence: 0°, 90°, 180°, 270°. For reference only



(/wiki/File:Pico_LCD_Rotate.jpg)

- Image Mirror Flip: Set the mirror flip of the selected image, you can choose no mirroring, horizontal mirroring, vertical mirroring, and mirroring around the image center

```
void Paint_SetMirroring(UBYTE mirror)
Parameter:
        mirror: The mirroring method of the image can be selected as MIRROR_NONE, MIRROR_HORIZONTAL, MIRROR_VERTICAL, and MIRROR_ORIGIN, which correspond to no mirroring, horizontal mirroring, vertical mirroring, and mirroring around the image center, respectively
```

- Set the position and color of the point to be displayed in the cache: This is the core function of the GUI, handling point position and color displayed in the cache;

```
void Paint_SetPixel(UWORD Xpoint, UWORD Ypoint, UWORD Color)
Parameter:
        Xpoint: The X position of the point in the image cache
        Ypoint: The Y position of the point in the image cache
        Color:  The color of the point
```

- Image cache fill color: Fill the image cache with a certain color, which is generally used as a screen whitening

```
void Paint_Clear(UWORD Color)
Parameter:
        Color: The filled color
```

- Image cache partial window fill color: Fill a certain part of the image cache window with a certain color, usually used as a window whitening function, commonly used for displaying time, whitening for one second

```
void Paint_ClearWindows(UWORD Xstart, UWORD Ystart, UWORD Xend, UWORD Yend, UWOR
D Color)
Parameter:
        Xstart: The starting point X coordinate of the window
        Ystart: The starting point Y coordinate of the window
        Xend:   The endpoint X coordinate of the window
        Yend:   The endpoint Y coordinate of the window
        Color:  The filled color
```

- Draw a dot: In the image cache, draw a dot at (Xpoint, Ypoint), you can choose the color, the size, and the style of the dot

```
void Paint_DrawPoint(UWORD Xpoint, UWORD Ypoint, UWORD Color, DOT_PIXEL Dot_Pixe
l, DOT_STYLE Dot_Style)
Parameter:
        Xpoint: The X coordinate of the dot
        Ypoint: The Y coordinate of the dot
        Color:  The filled color
        Dot_Pixel: The size of the dots, which provides the default 8 sizes of d
ots
                typedef enum {
                        DOT_PIXEL_1X1  = 1,           // 1 x 1
                        DOT_PIXEL_2X2  ,              // 2 X 2
                        DOT_PIXEL_3X3  ,              // 3 X 3
                        DOT_PIXEL_4X4  ,              // 4 X 4
                        DOT_PIXEL_5X5  ,              // 5 X 5
                        DOT_PIXEL_6X6  ,              // 6 X 6
                        DOT_PIXEL_7X7  ,              // 7 X 7
                        DOT_PIXEL_8X8  ,              // 8 X 8
                } DOT_PIXEL;
        Dot_Style: The style of the dot, the method to expand the size of the do
t is that, whether to expand from the dot as the center or from the dot as the l
ower left corner to the upper right
                typedef enum {
                   DOT_FILL_AROUND  = 1,
                   DOT_FILL_RIGHTUP,
                } DOT_STYLE;
```

- Draw a line: In the image cache, you can draw a line from (Xstart, Ystart) to (Xend, Yend) and choose the color, the width and the style of the line

```
void Paint_DrawLine(UWORD Xstart, UWORD Ystart, UWORD Xend, UWORD Yend, UWORD Co
lor, LINE_STYLE Line_Style , LINE_STYLE Line_Style)
Parameter:
        Xstart: The starting point X coordinate of the line
        Ystart: The starting point Y coordinate of the line
        Xend:   The endpoint X coordinate of the line
        Yend:   The endpoint Y coordinate of the line
        Color:  The filled color
        Line_width: The width of the lines, which provides the default 8 widths
                typedef enum {
                        DOT_PIXEL_1X1  = 1,           // 1 x 1
                        DOT_PIXEL_2X2  ,              // 2 X 2
                        DOT_PIXEL_3X3  ,              // 3 X 3
                        DOT_PIXEL_4X4  ,              // 4 X 4
                        DOT_PIXEL_5X5  ,              // 5 X 5
                        DOT_PIXEL_6X6  ,              // 6 X 6
                        DOT_PIXEL_7X7  ,              // 7 X 7
                        DOT_PIXEL_8X8  ,              // 8 X 8
                } DOT_PIXEL;
         Line_Style: The style of the line, which selects whether the line is co
nnected in a straight line or as a dashed line
                typedef enum {
                        LINE_STYLE_SOLID = 0,
                        LINE_STYLE_DOTTED,
                } LINE_STYLE;
```

- Draw a rectangle: In the image cache, you can draw a rectangle from (Xstart, Ystart) to (Xend, Yend) and choose the color, the width of the line, and whether to fill the inside of the rectangle

```
void Paint_DrawRectangle(UWORD Xstart, UWORD Ystart, UWORD Xend, UWORD Yend, UWO
RD Color, DOT_PIXEL Line_width, DRAW_FILL Draw_Fill)
Parameter:
        Xstart: The starting point X coordinate of the rectangle
        Ystart: The starting point Y coordinate of the rectangle
        Xend:   The endpoint X coordinate of the rectangle
        Yend:   The endpoint Y coordinate of the rectangle
        Color:  The filled color
        Line_width: The width of the four sides of the rectangle, providing the
default 8 widths
                typedef enum {
                        DOT_PIXEL_1X1  = 1,           // 1 x 1
                        DOT_PIXEL_2X2  ,              // 2 X 2
                        DOT_PIXEL_3X3  ,              // 3 X 3
                        DOT_PIXEL_4X4  ,              // 4 X 4
                        DOT_PIXEL_5X5  ,              // 5 X 5
                        DOT_PIXEL_6X6  ,              // 6 X 6
                        DOT_PIXEL_7X7  ,              // 7 X 7
                        DOT_PIXEL_8X8  ,              // 8 X 8
                } DOT_PIXEL;
        Draw_Fill: Filling, whether to fill the inside of the rectangle
                typedef enum {
                        DRAW_FILL_EMPTY = 0,
                        DRAW_FILL_FULL,
                } DRAW_FILL;
```

- Draw a circle: In the image cache, draw a circle with a radius of Radius with (X_Center Y_Center) as the center of the circle, and you can choose the color, the width of the line, and whether to fill the inside of the circle

```
void Paint_DrawCircle(UWORD X_Center, UWORD Y_Center, UWORD Radius, UWORD Color,
DOT_PIXEL Line_width, DRAW_FILL Draw_Fill)
Parameter:
        X_Center: The X coordinate of the center of the circle
        Y_Center: The Y coordinate of the center of the circle
        Radius:  The radius of the circle
        Color:    The filled color
        Line_width: The width of the arc, which provides the default 8 widths
                typedef enum {
                        DOT_PIXEL_1X1  = 1,           // 1 x 1
                        DOT_PIXEL_2X2  ,              // 2 X 2
                        DOT_PIXEL_3X3  ,              // 3 X 3
                        DOT_PIXEL_4X4  ,              // 4 X 4
                        DOT_PIXEL_5X5  ,              // 5 X 5
                        DOT_PIXEL_6X6  ,              // 6 X 6
                        DOT_PIXEL_7X7  ,              // 7 X 7
                        DOT_PIXEL_8X8  ,              // 8 X 8
                } DOT_PIXEL;
        Draw_Fill: Filling, whether to fill the inside of the circle
                typedef enum {
                        DRAW_FILL_EMPTY = 0,
                        DRAW_FILL_FULL,
                } DRAW_FILL;
```

- Write Ascii characters: In the image cache, with (Xstart Ystart) as the left vertex, write an Ascii character, and you can select the Ascii code visual character font library, font foreground color, and font background color

```
void Paint_DrawChar(UWORD Xstart, UWORD Ystart, const char Ascii_Char, sFONT* Fo
nt, UWORD Color_Foreground, UWORD Color_Background)
Parameter:
        Xstart: The left vertex X coordinate of the character
        Ystart: The left vertex Y coordinate of the character
        Ascii_Char: Ascii character
        Font: The Ascii code visual character font library, provides the followi
ng fonts in the Fonts folder:
                font8: 5*8 font
                font12: 7*12 font
                font16: 11*16 font
                font20: 14*20 font
                font24: 17*24 font
        Color_Foreground: Font color
        Color_Background: Background color
```

- Write English strings: In the image cache, write a string of English characters with (Xstart Ystart) as the left vertex, and you can select the Ascii code visual character font library, font foreground color, and font background color

```
void Paint_DrawString_EN(UWORD Xstart, UWORD Ystart, const char * pString, sFONT
* Font, UWORD Color_Foreground, UWORD Color_Background)
Parameter:
        Xstart:  The left vertex X coordinate of the character
        Ystart:  The left vertex Y coordinate of the character
        pString: A string, a string is a pointer
        Font: The Ascii code visual character font library, provides the followi
ng fonts in the Fonts folder:
                font8: 5*8 font
                font12: 7*12 font
                font16: 11*16 font
                font20: 14*20 font
                font24: 17*24 font
        Color_Foreground: Font color
        Color_Background: Background color
```

- Write Chinese strings: In the image cache, write a string of Chinese characters with (Xstart Ystart) as the left vertex, and you can select the GB2312 encoding character font library, font foreground color, and font background color

```
void Paint_DrawString_CN(UWORD Xstart, UWORD Ystart, const char * pString, cFONT
* font, UWORD Color_Foreground, UWORD Color_Background)
Parameter:
        Xstart:  The left vertex X coordinate of the character
        Ystart:  The left vertex Y coordinate of the character
        pString: A string, a string is a pointer
        Font: The GB2312 encoding character library, the following fonts are pro
vided in the Fonts folder:
                font12CN: Ascii character font 11*21, Chinese font 16*21
                font24CN: Ascii character font 24*41, Chinese font 32*41
        Color_Foreground: Font color
        Color_Background: Background color
```

- Write numbers: In the image cache, write a string of numbers with (Xstart Ystart) as the left vertex, and you can select the Ascii code visual character font library, font foreground color, and font background color

```
void Paint_DrawNum(UWORD Xpoint, UWORD Ypoint, double Nummber, sFONT* Font, UWOR
D Digit,UWORD Color_Foreground, UWORD Color_Background);
Parameter:
        Xstart:  The left vertex X coordinate of the character
        Ystart:  The left vertex Y coordinate of the character
        Number: The displayed number, here is stored in a 32-bit long int type,
which can be displayed up to 2147483647.
        Font: The Ascii code visual character font library, provides the followi
ng fonts in the Fonts folder:
                font8: 5*8 font
                font12: 7*12 font
                font16: 11*16 font
                font20: 14*20 font
                font24: 17*24 font
        Digit: Display decimal places
        Color_Foreground: Font color
        Color_Background: Background color
```

- Display time: In the image cache, display time with (Xstart Ystart) as the left vertex, and you can select the Ascii code visual character font library, font foreground color, and font background color

```
void Paint_DrawTime(UWORD Xstart, UWORD Ystart, PAINT_TIME *pTime, sFONT* Font,
UWORD Color_Background, UWORD Color_Foreground)
Parameter:
        Xstart: The left vertex X coordinate of the character
        Ystart: The left vertex Y coordinate of the character
        pTime:  The displayed time, here defines a time structure, just pass the
numbers of the hour, minute, second to the parameters
        Font: The Ascii code visual character font library, provides the followi
ng fonts in the Fonts folder:
                font8: 5*8 font
                font12: 7*12 font
                font16: 11*16 font
                font20: 14*20 font
                font24: 17*24 font
        Color_Foreground: Font color
        Color_Background: Background color
```

Python

Underlying Hardware Interface

- Module initialization

```
def __init__(self)
```

- Send a command

```
def write_cmd(self, cmd)
```

- Send data

```
def write_data(self, buf)
```

- Adjust backlight

```
def set_bl_pwm(self,duty)
```

Drawing GUI

- Import a library

```
import framebuf
```

- Create an object

The custom class LCD_1inch28 inherits the framebuf.FrameBuffer class from MicroPython, which provides many methods to draw images, let's create the LCD_1inch28 class object first

```
LCD = LCD_1inch28()
```

- Draw a line

```
LCD.line(x1, y1, x2, y2, color)
Parameter:
        x1, y1: The x and y coordinates of the starting point
        x2, y2: The x and y coordinates of the endpoint
        color:  The color of the line
```

- Draw a rectangle

```
LCD.fill_rect(x1, y1, w, h, color)
Parameter:
        x1, y1: The x and y coordinates of the upper left corner of the rectangl
e
        w, h:   The width and height of the rectangle
        color:  The filling color of the rectangle
```

- Draw text

```
LCD.text(str, x, y, color)
Parameter:
         str:   Display text
         x, y:  The x and y coordinates of the upper left corner of the text
         color: The color of the text
```

- Change the text size

```
LCD.write_text(str, x, y, size, color)
Parameter:
         size: The difference between this function and LCD.text is that it supp
orts custom font size, which is used to specify the font size
```

- Display

```
LCD.show()
```

Demo

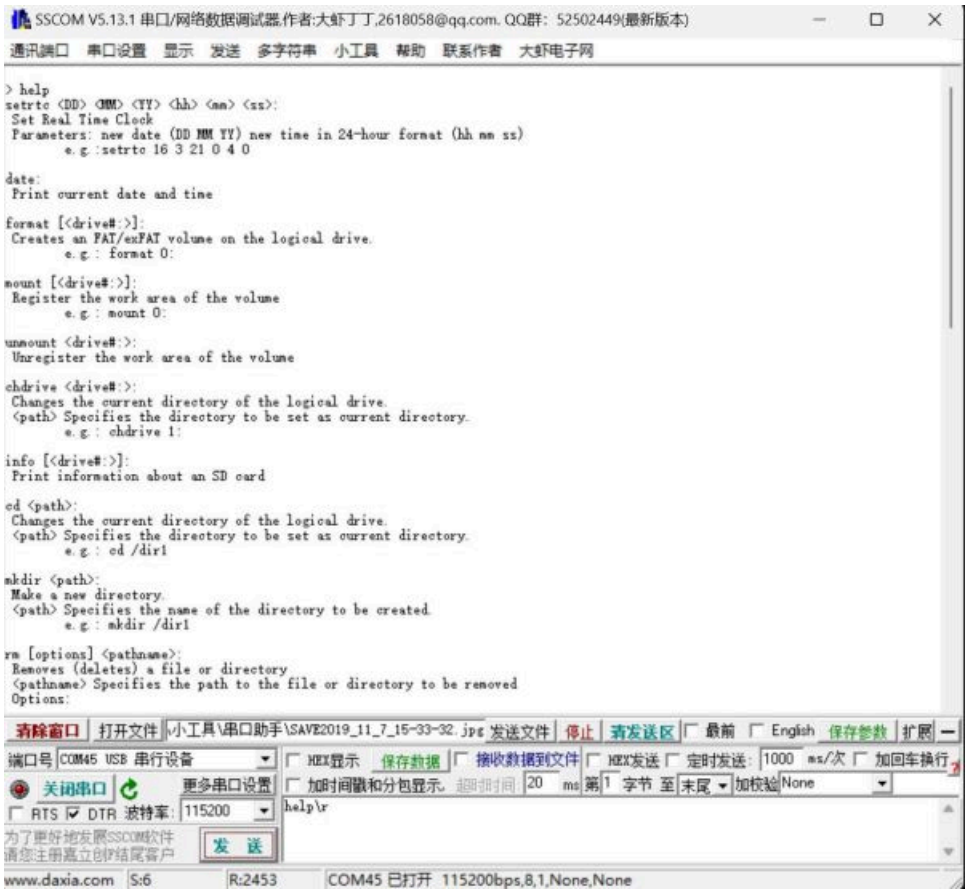
C/C++ Demo

01-LCD

- Example of LCD running GUI

02-FATFS

- The demo is based on the open-source program no-OS-FatFS-SD-SDIO-SPI-RPi-Pico (https://github.com/carlk3/no-OS-FatFS-SD-SDIO-SPI-RPi-Pico)
    - The demo implements a command-line interface similar to busybox or DOS. It can mount a TF card, create and view files, and so on. The command "help" can be used to view all available commands



(/wiki/File:Geek-code-00.jpg)

- Note that the TF card file format is FAT32

03-RGB

- Example of controlling onboard RGB LEDs

MicroPython Demo

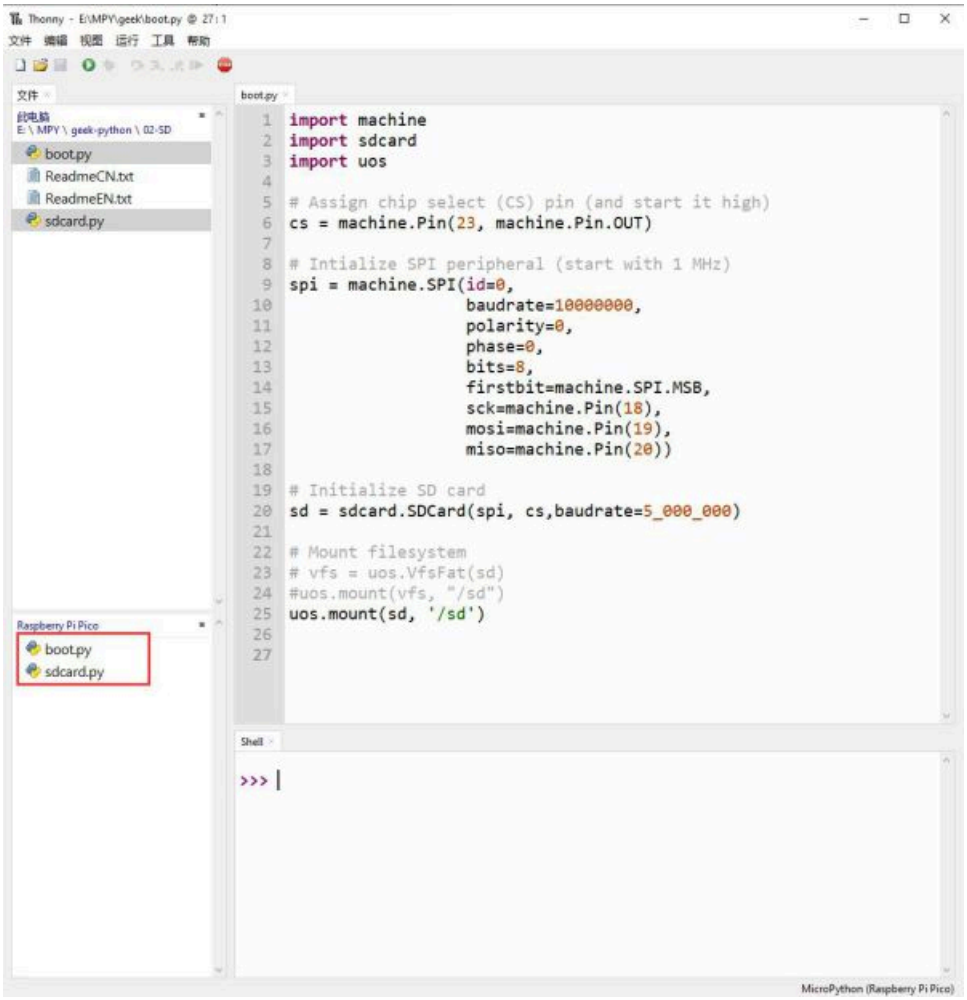
01-LCD

- Example of LCD running GUI

02-SD

How to Use

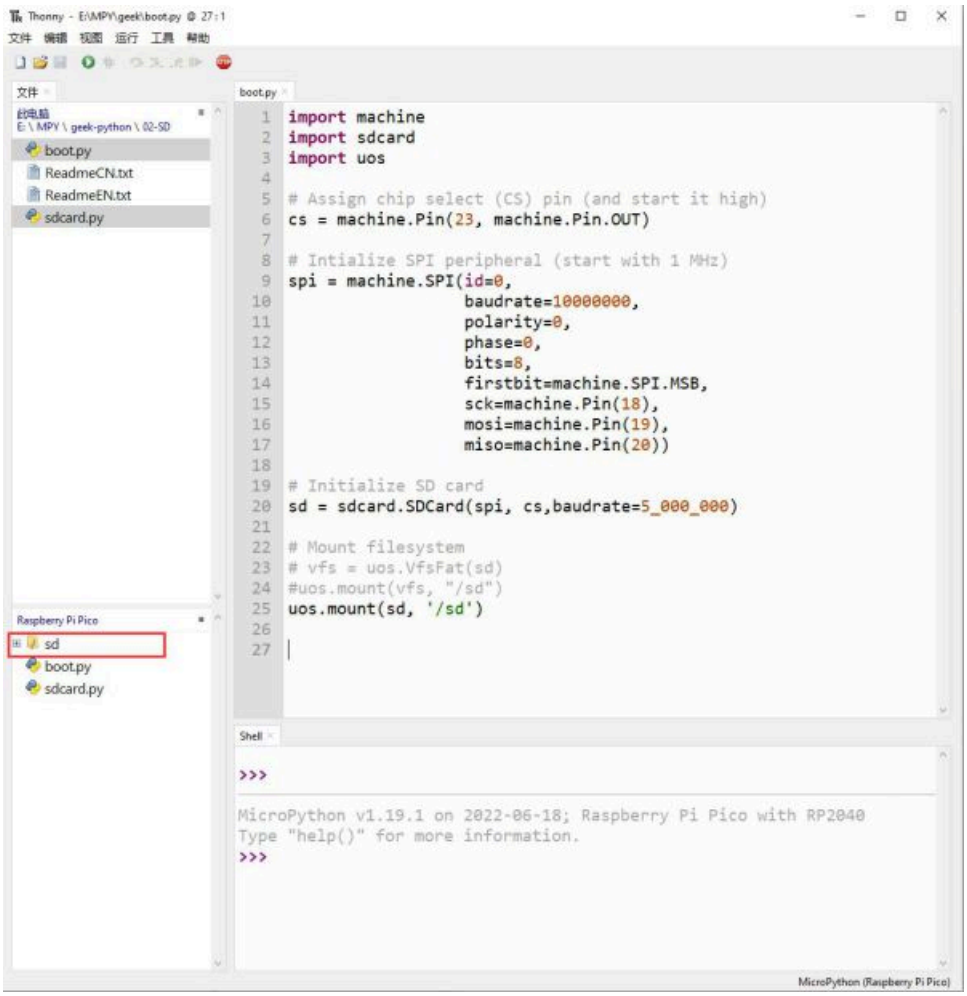
- Upload all the .py files in the file to the development board via Thonny and reset (the image is for demonstration only, please refer to the example for SD communication pins)



(/wiki/File:Geek-code-02.jpg)

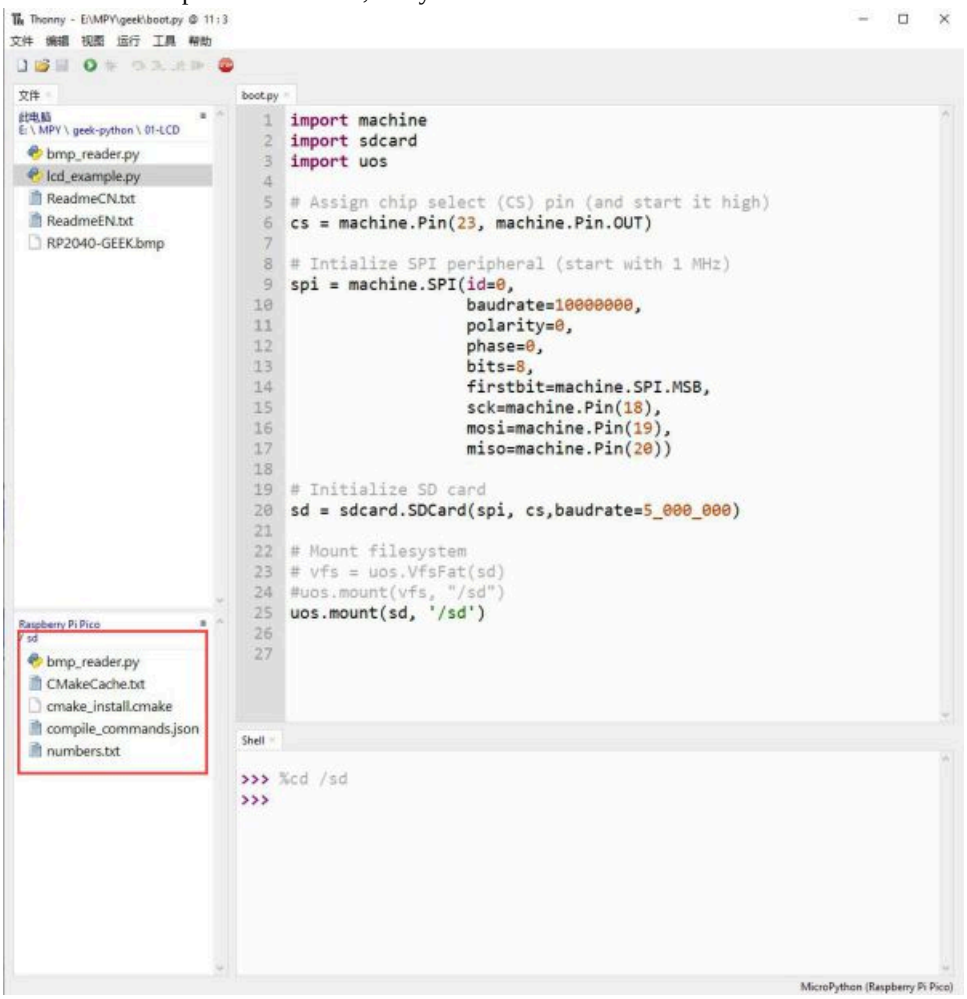
Running Result

- After the development board is reset, it will automatically mount the TF to the SD folder according to the boot.py program



(/wiki/File:Geek-code-03.jpg)

- Double-click to open the SD folder, and you can see the files stored in the TF card



(/wiki/File:Geek-code-04.jpg)

03-RGB

- Example of controlling onboard RGB LEDs

CircuitPython Demo

Directory Structure

```
├── examples # Demos
│   ├── rename_device_script.py # Example of renaming a device
│   ├── Display_images # This directory contains an example of displaying images
on the RP2350-LCD-1.47
│   │   ├── Display_Images_From_SDCard # Read images from the TF card and displa
y them
│   │   └── Display_Images_From_Board # Read images from RP2350-LCD-1.47 and dis
play them
│   ├── windows # This directory contains examples of RP2350-LCD-1.47 on Windows
operating system
│   │   ├── 01_AccessCamera.py # Turn on the camera
│   │   ├── 02_CoolFake_Hacking.py # Simulate hacker attacks
│   │   ├── 03_CreateFile.py # Create a file
│   │   ├── 04_DeleteBrowsing_History.py # Delete browser history
│   │   ├── 05_Mouse_control.py # Control mouse movements
│   │   ├── 06_openWebsite.py # Open the webpage
│   │   ├── 07_PC_Restart.py # Restart the computer
│   │   ├── 08_PC_Shutdown.py # Turn off the computer
│   │   ├── 09_RunNetwork_commands.py # Execute network-related commands
│   │   └── 10_Youtube_Channel_Subscribe.py # Subscribe to the Youtube channel
│   └── mac # This directory contains examples of RP2350-LCD-1.47 on the MAC ope
rating system
│       ├── 01_AccessCamera.py # Turn on the camera
│       ├── 02_CoolFake_Hacking.py # Simulate hacker attacks
│       └── ......(Same functionality as Windows example)
├── lib # Library file, after flashing firmware, upload the entire lib directory
to the development board
├── firmware.uf2 # CircuitPython version 10.0.3 firmware
└── LICENSE
```

How to Use

1. Flash the firmware.uf2 firmware
2. After Flashing the firmware, upload the entire CircuitPython\lib directory to the board
3. After uploading, you can directly run any program by selecting it in the Thonny IDE
4. Run scripts independently without Thonny IDE
    - In Thonny IDE, select the program to run automatically. In the top left corner, go to File → Save As → CircuitPython Device → Select code.py → Confirm
    - If the filename is not code.py, the device will not automatically execute the script after a reboot or reinsertion of the USB
    - After copying is complete, remove the device from the USB port and reinsert it into the computer
    - After the device is powered on again, code.py will be executed automatically without needing to start the Thonny IDE or perform any additional operations

LVGL Demos

C

Example Demonstration

The effect displayed in this example is to show an image through LVGL.

Demo Introduction

This example is used to test the LVGL widget interaction and style beautification, etc.. For specific development of LVGL, please refer to LVGL development documentation (https://docs.lvgl.io/8.1/).

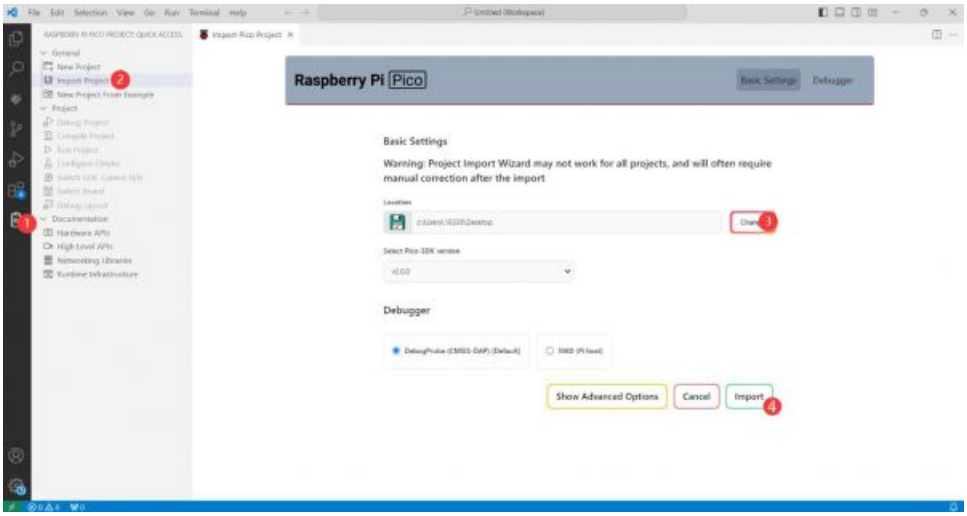
Implement Function

1. In this example, DMA is used to transfer color data to the SPI bus, reducing CPU utilization, and controlling CPU usage to less than 50% and memory usage to less than 35% during simple interactions.
2. The example system clock is 200MHz. The peripheral clock frequency for SPI is set to match the system clock, and the LVGL library's dual buffer mechanism is utilized. Data transfer occurs in one buffer while the other buffer is rendering, ensuring smooth animation.
3. This example displays an image through LVGL, demonstrating the simple use of LVGL widget.

Compile and Run

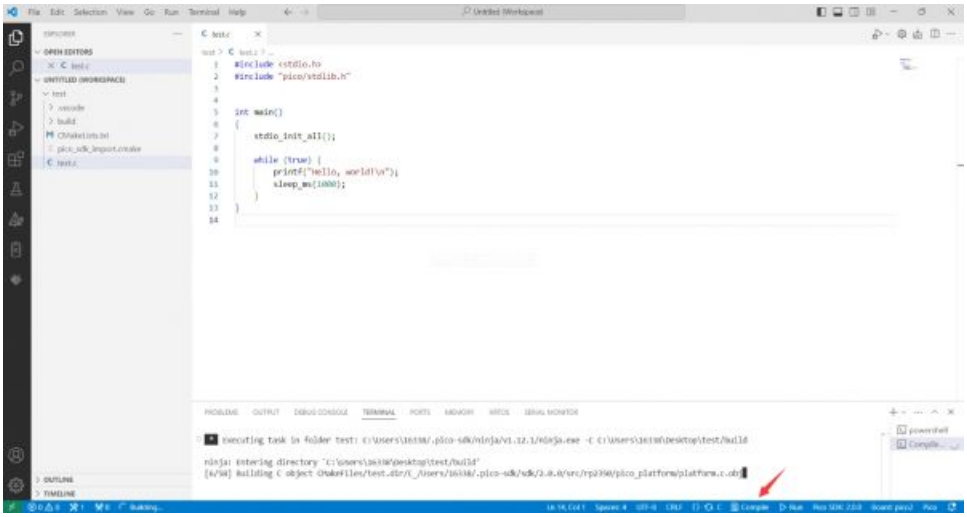
- Pico VSCode

1. Select the project directory and import the project



(/wiki/File:600px-Pico-vscode-23.jpg)

2. Select the version and SDK version, click Compile to compile



(/wiki/File:Pico-vscode-19.JPG)

- Windows
  - Refer to Windows Environment Setup Tutorial (https://www.waveshare.com/wiki/Pico-Get-Start-Windows) to complete the environment setup
  - Open VS 2022 -> Tool -> Command Line -> Developer Powershell
  - Set the absolute address of pico-sdk as PICO_SDK_PATH, for example, set pico-sdk address as "D:\pico\pico-sdk"

    ```
    setx PICO_SDK_PATH "D:\pico\pico-sdk"
    ```

  - Download the demo, enter the source code directory, if the **build** directory already exists, you can go directly into it. If not, you can create this directory:

    ```
    mkdir build
    cd build
    ```

  - Execute **cmake**, automatically generate the **Makefile** file:

    ```
    cmake -G "NMake Makefiles" ..
    ```

  - Execute **nmake** to generate the executable file, and input the following content in the terminal:

    ```
    nmake
    ```

    After compilation, it will generate a **.uf2** formatted file.
  - Press the onboard boot key, connect the board to the USB interface of the PC through a Micro USB cable. And then release the key, the PC will identify the pico as a removable driver. Finally, you need to copy the compiled file in .uf2 format to Pico.
- Ubuntu
  - Refer to Chapter 2. The SDK of Getting started with Raspberry Pi Pico (https://files.waveshare.com/upload/3/30/Getting_started_with_pico.pdf) to complete the environment setup
  - Open a terminal, set the value of the environment variable PICO_SDK_PATH to the absolute path of pico-sdk, for example, if my pico-sdk path is "/home/pico/pico-sdk"

    ```
    nano ~/.bashrc
    #Add the following content at the last line
    export PICO_SDK_PATH="/home/pico/pico-sdk"
    ```

  - After setting, save and exit. The configuration takes effect

```
source ~/.bashrc
```

- Download the demo, enter the source code directory, if the **build** directory already exists, you can go directly into it. If not, you can create this directory:

```
mkdir build
cd build
```

- Execute **cmake**, it will generate **Makefile** file:

```
cmake ..
```

- Execute **nmake** to generate the executable file, and input the following content in the terminal:

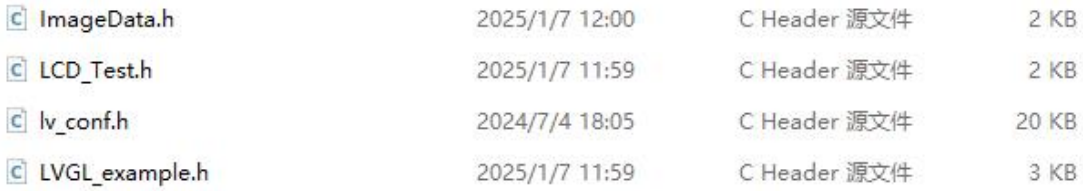
```
nmake
```

  After compilation, it will generate a **.uf2** formatted file.
- Press the onboard boot key, connect the board to the USB interface of the PC through a Micro USB cable. And then release the key, the PC will identify the pico as a removable driver. Finally, you need to copy the compiled file in .uf2 format to Pico.

Source Code Analysis

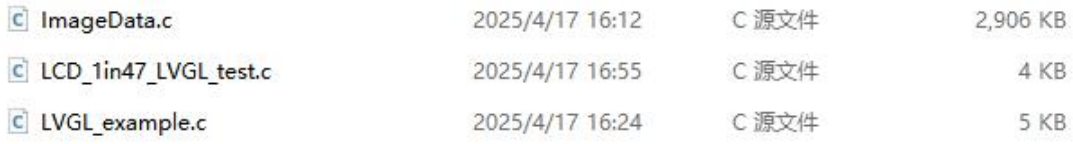
Source Code Structure

- The source code of the LVGL library is at **lib/lvgl** of the project file folder, and the version used is "8.1". For secondary development, please refer to the development documentation corresponding to the used version.
- The related settings for the LVGL library are at **examples/inc/lv_conf.h** of the project file folder, where you can set display refresh rates, system usage data, and so on.



(/wiki/File:RP2350-GEEK-LVGL-inc.jpg)

- The application code for the LVGL library is located in the project folder at **examples/src/LVGL_example.c**.



(/wiki/File:RP2350-LCD-1.47-LVGL-src.jpg)

LVGL Initialization

Before using LVGL image library, you need to initialize LVGL.

- The initialization function for the LVGL library
  - Demo location: **examples/src/LVGL_example.c**
  - Implementation function: Mainly used to initialize the hardware and structure variables required for LVGL.

```
LVGL_Init();
```

- LVGL library core initialization

- Demo location: **examples/src/LVGL_example.c**

```
/*2.Init LVGL core*/
lv_init();
```

LVGL Run

The LVGL library calls the heartbeat function lv_tick_inc at regular intervals to notify LVGL of the elapsed time so that LVGL can update its internal time state and handle time-related tasks such as animations, timers, etc. The lv_task_handler function also needs to be called in the loop of the main function so that LVGL can handle events and tasks in time to ensure that the user interface responds and refreshes.

- LVGL heartbeat interface
  - Demo location: **examples/src/LVGL_example.c**
  - Implementation method: It is necessary to ensure that the priority of **lv_task_handler** is lower than the priority of **lv_tick_inc**. Therefore, in this example, **lv_tick_inc** is called within the timer callback function.

```
//Timer callback function called every 5ms
add_repeating_timer_ms(5, repeating_lvgl_timer_callback, NULL, &lvgl_timer);

static bool repeating_lvgl_timer_callback(struct repeating_timer *t)
{
    lv_tick_inc(5);
    return true;
}
```

- LVGL task processor
  - Demo location: **examples/src/LCD_XinXX_LVGL_test.c**
  - Implementation method: To handle LVGL tasks, you need to regularly call **lv_timer_handler()**, in this example, it is called in the loop of the main function.

```
int main()
{
    ...
    while (1)
    {
      lv_task_handler();
      DEV_Delay_ms(5);
      ...
    }

}
```

LVGL Display

To implement LVGL display, you must initialize a display driver and set the various properties of the display driver, such as color format, draw buffer, rendering mode, and display callback function. At each LV_DISP_DEF_REFR_PERIOD (set in lv_conf.h), LVGL detects if something has happened on the UI that needs to be redrawn. For example, a button is pressed, a chart is changed, an animation occurs, etc. When redrawing is needed, LVGL calls the display callback function to complete the drawing of the image in the refresh area.

- LVGL display refreshing rate setting
  - Demo location: **examples/inc/lv_conf.h**
  - Setting method: In the **lv_conf.h** file, you can also set the display buffer refresh rate. You can modify this definition to change the screen refresh time.

```
#define LV_DISP_DEF_REFR_PERIOD  10 // Unit: ms, here is 10ms
```

- LVGL display color setting
  - Demo location: **examples/inc/lv_conf.h**
  - Purpose of setting: Since the pixel color storage method constructed by the **lv_color_t** structure in its default state does not match the data to be transmitted in this example, direct transmission would result in color discrepancies in the displayed image.

```
#define LV_COLOR_16_SWAP 1
```

- LVGL display related variable definition
  - Demo location: **examples/src/LVGL_example.c**
  - Implementation function: Define the display driver disp_drv, draw the buffer disp_buf. This example implements a double buffering mechanism, with the drawing buffer composed of buffers buf0 and buf1, both set to half the size of the screen display area, which can reduce jagged edges during large area refreshing while effectively improving screen refresh rate; when using a single buffer, it is best to set it to 10% of the screen display area, which can effectively reduce system usage but may result in noticeable jagged edges during large area refresh of image.

```
static lv_disp_drv_t disp_drv;
static lv_disp_draw_buf_t disp_buf;
static lv_color_t buf0[DISP_HOR_RES * DISP_VER_RES/2];
static lv_color_t buf1[DISP_HOR_RES * DISP_VER_RES/2];
```

- LVGL display device registration
  - Demo location: **examples/src/LVGL_example.c**
  - Implementation function: According to the design requirements, improve the core structure variables of the LVGL library, initialize the display driver disp_drv, and set the drawing buffer, which is a simple array used by LVGL to render screen content. Once the rendering is ready, the content of the draw buffer will be sent to the display using the disp_drv_flush_cb function set in the display driver.

```
/*3.Init LVGL display*/
lv_disp_draw_buf_init(&disp_buf, buf0, buf1, DISP_HOR_RES * DISP_VER_RES /
2);
lv_disp_drv_init(&disp_drv);
disp_drv.flush_cb = disp_flush_cb;
disp_drv.draw_buf = &disp_buf;
disp_drv.hor_res = DISP_HOR_RES;
disp_drv.ver_res = DISP_VER_RES;
lv_disp_t *disp= lv_disp_drv_register(&disp_drv);
```

- LVGL display callback function
  - Demo location: **examples/src/LVGL_example.c**
  - Implementation function: Mainly complete the drawing of the image in the refresh area.

```
void disp_flush( lv_disp_drv_t *disp_drv, const lv_area_t *area, lv_color_t
*color_p )
Parameter:
    lv_disp_drv_t *disp_drv: Displays driver structure pointers, which conta
in information about the display and function pointers. This parameter is of
ten used to notify you that a refresh is complete
    const lv_area_t *area  : Region structure pointer, containing the positi
on information of the area to be refreshed. In this demo, you can use it for
creating TFT display window.
    lv_color_t *color_p    : Color structure pointer, indicating the color d
ata to be displayed in the refresh area. In this demo, it reads the address
as DMA input to transmit data to the SPI bus and completes the image drawing
```

- LVGL display callback function implementation
  - Demo location: **examples/src/LVGL_example.c**
  - Implementation method: In this example, to maximize the reduction of processor utilization, DMA is used for color data transmission. **color_p** is set as the read address, and the SPI bus output data register is set as the write address.

```
static void disp_flush_cb(lv_disp_drv_t * disp, const lv_area_t * area, lv_c
olor_t * color_p)
{
    LCD_SetWindows(area->x1, area->y1, area->x2+1, area->y2+1);
    DEV_Digital_Write(LCD_DC_PIN, 1);
    DEV_Digital_Write(LCD_CS_PIN, 0);
    dma_channel_configure(dma_tx,
                          &c,
                          &spi_get_hw(LCD_SPI_PORT)->dr,
                          color_p, // read address
                          ((area->x2 + 1 - area->x1) * (area->y2 + 1 - area-
>y1))*2,
                          true);
}
```

- LVGL refresh completion notification implementation
  - Demo location: **examples/src/LVGL_example.c**
  - Implementation function: The LVGL core needs to be notified after each image refresh is completed, so that LVGL can prepare the next refresh image for rendering.
  - Implementation method: This example notifies LVGL image refresh completion in the DMA transfer complete interrupt service function. If a blocking notification mechanism is used, it is not possible to utilize the double buffering mechanism to increase the refresh rate.

```
static void dma_handler(void)
{
    if (dma_channel_get_irq0_status(dma_tx)) {
        dma_channel_acknowledge_irq0(dma_tx);
        DEV_Digital_Write(LCD_CS_PIN, 1);
        lv_disp_flush_ready(&disp_drv);
    }
}
```

LVGL Widget Layout

In LVGL, we can create various user interfaces. The basic components of the interface are objects, also called widgets, such as buttons, labels, images, lists, charts, or text areas. In a interface, we can create multiple widgets simultaneously and set their positions, sizes, parent objects, styles, and event handlers and other basic properties.

- LVGL widget initialization
  - Demo location: **examples/src/LVGL_example.c**
  - Function: It primarily used for stylized widgets and layout widgets.

```
void Widgets_Init(void);
```

- LVGL create tile
  - Demo location: **examples/src/LVGL_example.c**
  - Implementation function: A Tile view is a container object whose elements, called tiles, can be arranged in a grid format. Users can navigate between tiles by swiping. Call lv_tileview_add_tile(tileview, row_id, col_id, dir) to create a new tile at row_id row and col_id column. dir can be LV_DIR_LEFT/RIGHT/TOP/BOTTOM/HOR/VER/ALL or a value to move to the adjacent tile in the given direction by sliding.

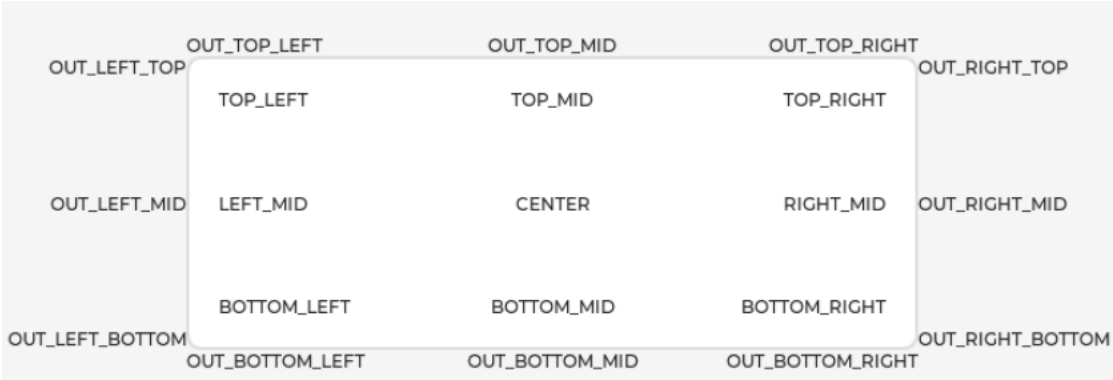
```
//Create a tile at (0,0) with support for sliding down to (0,1)
tile1 = lv_tileview_add_tile(tv, 0, 0, LV_DIR_BOTTOM);
```

- LVGL create widget
  - Demo location: **examples/src/LVGL_example.c**
  - Implementation function: To create a widget, different widgets need to use different function interfaces, you can choose the parent object to create it.

```
//Create a widget where tab1 is the parent object of the key, which can be r
eplaced with a widget such as list, title, etc. that can have child objects
sw = lv_switch_create(tab1);
```

- Alignment positioning of LVGL widget
  - Demo location: **examples/src/LVGL_example.c**
  - Function: Enable a widget to be offset and positioned based on a reference point. The widget aligns to the center of the offset reference point widget.
  - Alignment standard: The LVGL library supports both internal and external alignments. By default, the upper-left corner is the origin, the leftward as the positive horizontal direction, and the downward as the positive vertical direction.

```
//Position the btn widget 45 pixels to the left from the center point
lv_obj_align(sw, LV_ALIGN_CENTER, -45, 0);
```



(/wiki/File:RP2040-Touch-LCD-1.28-LVGL-align.png)

- LVGL widget adjust font size
  - Demo location: **examples/inc/lv_conf.h** and **examples/src/LVGL_example.c**
  - Implementation function: In actual use, an interface may need to use multiple font sizes. Multiple font sizes can be enabled in **lv_conf.h** and the default font size can be set. When setting the font size, it is necessary to stylize the widget so that it can be rendered according to the set style. The **lv_obj_add_style** function can be used to render various parts of the widget in different states.

```
#define LV_FONT_MONTSERRAT_16 1                                    // Enable fon
t 16
#define LV_FONT_MONTSERRAT_18 1                                    // Enable fon
t 18
#define LV_FONT_DEFAULT &lv_font_montserrat_18                     // Set the de
fault font size as 18

static lv_style_t style_label;
lv_style_init(&style_label);                                       // Initialize
style
lv_style_set_text_font(&style_label, &lv_font_montserrat_16);  // Set the fo
nt size as 16
lv_obj_add_style(label,&style_label,0);                            // Set label
theme style
```

- LVGL widget event handling
  - Demo location: **examples/src/LVGL_example.c**
  - Function: In LVGL, you can add an event handler callback function to a widget so that when the widget is clicked, scrolled, redrawn, etc., it triggers an event and enters the event handler callback function. Call lv_obj_add_event_cb(obj, event_cb, filter, user_data) function in the program to add the event filter handling function event_cb for the widget obj, when the widget obj triggers the filter event, the system will automatically call the event_cb function. The last parameter is a pointer to any custom data available in the event.

```
//Add handler function sw_event_cb for event LV_EVENT_VALUE_CHANGED to widge
t sw
lv_obj_add_event_cb(sw, sw_event_cb,LV_EVENT_VALUE_CHANGED,NULL);
```

Python

Example Demonstration

The effect displayed in this example is to show an image through LVGL.

Demo Introduction

This demo is for testing LVGL widget interaction, style design and so on. The version used is 9.1. As the development document of LVGL V9 does not provide Python example, you can refer to LVGL development documentation (https://docs.lvgl.io/8.1/).

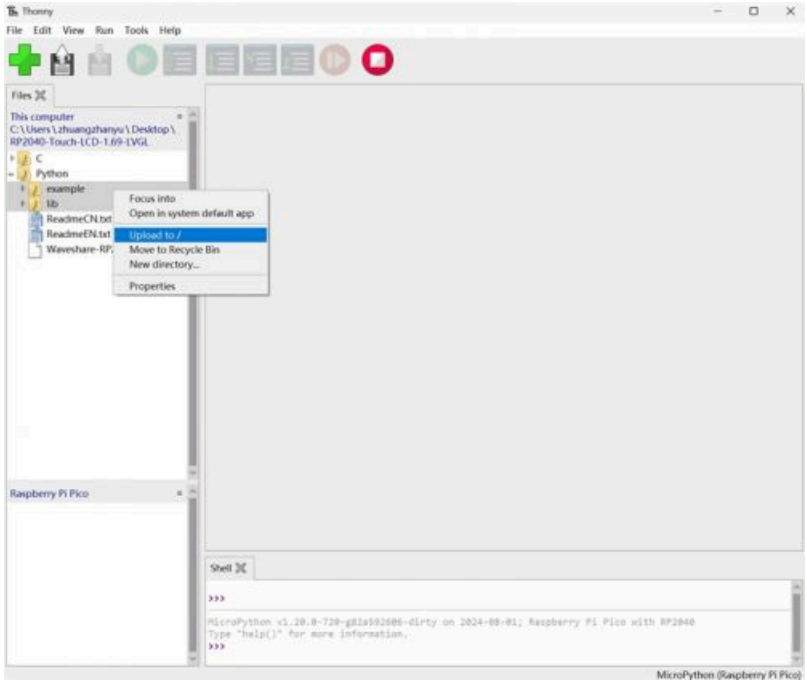
Implement Function

1. The example system clock is 230MHz. The peripheral clock frequency for SPI is set to match the system clock, and the LVGL library's dual buffer mechanism is utilized. Data transfer occurs in one buffer while the other buffer is rendering, ensuring smooth animation.
2. This example displays an image through LVGL, demonstrating the simple use of LVGL widget.

Compile and Run

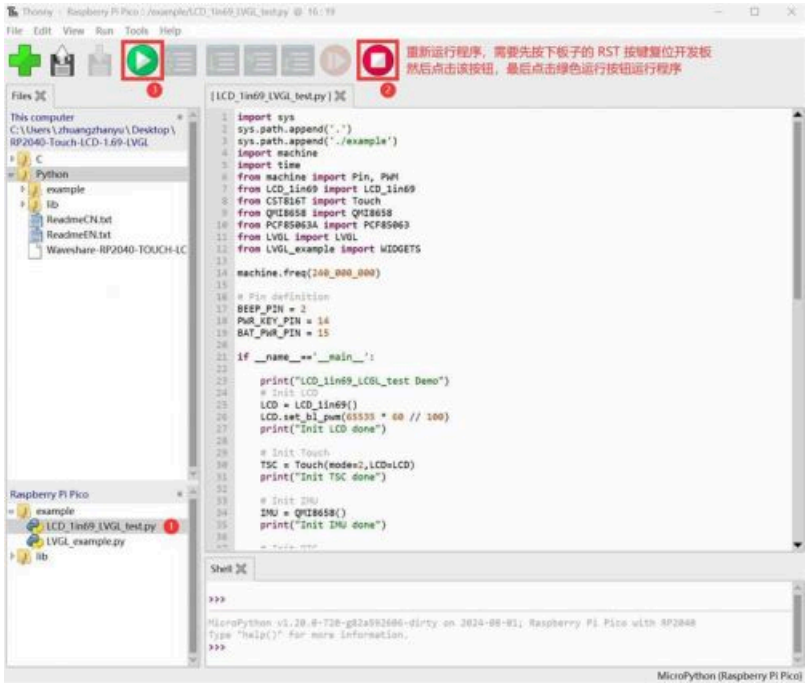
- Operation steps
- Before starting to use, it is necessary to set up an integrated development environment Thonny Python IDE (Windows version V3.3.3) (https://files.waveshare.com/wiki/common/Thonny-3.3.3.zip)
  - Download the demo, and press the onboard boot key, connect the board to the USB interface of the PC through a Micro USB cable. And then release the key, the PC will identify the pico as a removable driver. Finally, you need to copy the compiled file in .uf2 format to Pico.

- Open Thonny, upload the **examples** and **lib** directories to the development board



(/wiki/File:RP2040-Touch-LCD-1.69-LVGL-PY-1.jpg)

- Open the **LCD_1in47_LVGL_test.py** file under the **examples** directory, please refer to the steps below to run the demo



(/wiki/File:RP2040-Touch-LCD-1.69-LVGL-PY-2.jpg)

- Firmware building
  - Envitonment building: only test on Ubuntu 22.04
  - Github: lv_micropython (https://github.com/lvgl/lv_micropython.git)
  - Refer to Raspberry Pi Pico port (https://github.com/lvgl/lv_micropython?tab=readme-ov-file#raspberry-pi-pico-port) to compile
  - Firmware generation path: **lv_micropython/ports/rp2/build-PICO/firmware.uf2**

Source Code Analysis

Source Code Structure

- For LVGL library source code, you can refer to lv_micropython (https://github.com/lvgl/lv_micropython.git)

- The related setting of the LVGL library is at **lv_micropython/lib/lv_bindings/lv_conf.h** file folder, which sets the refreshing frequency of the display, system occupied data and so on.
- The application code for the LVGL library is located in **examples/src/LVGL_example.c** and **lib/LVGL.py** folders of the project folder.

LCD_1in47_LVGL_test.py	2025/4/18 9:42	Python file	1 KB
LVGL_example.py	2025/4/18 9:54	Python file	433 KB
LCD_1in47.py	2025/4/18 9:43	Python file	4 KB
lv_utils.py	2025/4/18 9:43	Python file	6 KB
LVGL.py	2025/4/18 9:43	Python file	3 KB

(/wiki/File:RP2350-LCD-1.47-LVGL-PY-3.jpg)

LVGL Initialization

Before using LVGL image library, you need to import the LVGL library and initialize the LVGL objects

- Import an LVGL library
  - Demo location: **lib/LVGL.py**
  - Function: Import LVGL library and use alias "lv" to facilitate calling its functions

```
import lvgl as lv
```

- Create an LVGL object
  - Demo location: **examples/LCD_1in47_LVGL_test.py**
  - Function: Create an LVGL object and pass in LCD and TSC objects as parameters

```
# Init LVGL
LVGL(LCD=LCD,TSC=TSC)
```

- LVGL initialization
  - Demo location: **lib/LVGL.py**
  - Code description: LVGL core initializes the initialization function located in the LVGL class, which is automatically called when the LVGL object is created

```
if not lv.is_initialized(): lv.init()
```

LVGL Run

The LVGL library periodically calls the heartbeat interface function lv.tick_inc to inform LVGL of the past time so that LVGL can update its internal time state to handle time-related tasks such as animations, timers, and so on. In addition, the lv.task_handler function needs to be called so that LVGL can handle events and tasks in time to ensure that the user interface responds and refreshes.

- LVGL Running interface
  - Demo location: **lib/LVGL.py**
  - Implementation method: Create an event_loop object. In the initialization function of this class, we create a timer. The timer will automatically call the heartbeat interface function and event handler function within a set time interval. The time interval for the call can be adjusted by passing the freq parameter, such as lv_utils.event_loop(freq=200), which defaults to 40 ms.

```
# Create event loop if not yet present
if not lv_utils.event_loop.is_running(): self.event_loop=lv_utils.event_loop
()
```

- LVGL heartbeat interface

- Demo location: **lib/lv_utils.py**
- Implementation method: You need to make sure that the priority of **lv.task_handler** is lower than that of **lv.tick_inc** , so in this case, **lv.tick_inc** is called in the timer callback function.

```
#Timer callback function called every 5ms
self.timer.init(mode=Timer.PERIODIC, period=self.delay, callback=self.timer_
cb) // In this case, self.delay = 5

def timer_cb(self, t):
    lv.tick_inc(self.delay)
    if self.scheduled < self.max_scheduled:
        try:
            micropython.schedule(self.task_handler_ref, 0)
            self.scheduled += 1 # The number of tasks being processed has in
creased
        except:
            pass
```

- LVGL task processor
  - Demo location: **lib/lv_utils.py**
  - Implementation method: To handle LVGL tasks, you need to call **lv.task_handler** at regular intervals, in this case after calling **lv.tick_inc** in the timer callback function.

```
def task_handler(self, _):
    try:
        if lv._nesting.value == 0:
            lv.task_handler()
            if self.refresh_cb: self.refresh_cb()
        self.scheduled -= 1 # The number of tasks being processed has decrea
sed
    except Exception as e:
        if self.exception_sink:
            self.exception_sink(e)
```

LVGL Display

To implement LVGL display, you must initialize a display driver and set the various properties of the display driver, such as color format, draw buffer, rendering mode, and display callback function. At each LV_DEF_REFR_PERIOD (set in lv_conf.h), LVGL detects if something has happened on the UI that needs to be redrawn. For example, a button is pressed, a chart is changed, an animation occurs, etc. When redrawing is needed, LVGL calls the display callback function to complete the drawing of the image in the refresh area.

- LVGL display refreshing rate setting
  - Demo location: **lv_micropython/lib/lv_bindings/lv_conf.h**
  - Setting method: In **lv_conf.h**, you can modify LV_DEF_REFR_PERIOD to change the refresh time of the screen.

```
#define LV_DEF_REFR_PERIOD 10 // Unit: ms, here is 10ms
```

- LVGL display related variable definition
  - Demo location: **lib/LVGL.py**
  - Function: The sizes of buf0 and buf1 are set to 33% of the screen display area to implement the LVGL dual buffer mechanism, which reduces the appearance of jagged edges during large area refresh of screen updates while effectively increasing the screen refresh rate; when using a single buffer, it is best to set it to 10% of the screen display area, which can effectively reduce system usage but may result in noticeable jagged edges during large area refresh of image.

```
# Init LVGL display
buf1 = lv.draw_buf_create(self.LCD.width, self.LCD.height // 3 , color_forma
t, 0)
buf2 = lv.draw_buf_create(self.LCD.width, self.LCD.height // 3 , color_forma
t, 0)
```

- LVGL display device registration
  - Demo location: **lib/LVGL.py**
  - Implementation function: According to the design requirements, improve the core structure variables of the LVGL library, initialize the display driver disp_drv, and set the drawing buffer, which is a simple array used by LVGL to render screen content. Once the rendering is ready, the content of the draw buffer will be sent to the display using the disp_drv_flush_cb function set in the display driver.

```
self.disp_drv = lv.display_create(self.LCD.width, self.LCD.height) # Create
a display driver object and set the width and height
self.disp_drv.set_color_format(color_format) # Set color format to RGB565
self.disp_drv.set_draw_buffers(buf1, buf2)   # Set the drawing buffer
self.disp_drv.set_render_mode(lv.DISPLAY_RENDER_MODE.PARTIAL) # Set the rend
ering mode to partial refresh mode
self.disp_drv.set_flush_cb(self.disp_drv_flush_cb) # Set display callback fu
nction
```

- LVGL display callback function
  - Demo location: **lib/LVGL.py**
  - Implementation function: Mainly complete the drawing of the image in the refresh area.

```
def disp_drv_flush_cb(self,disp_drv,area,color_p)
Parameter:
    disp_drv : Displays driver structure pointers, which contain information
about the display and function pointers. This parameter is often used to not
ify you that a refresh is complete
    area     : Region structure pointer, containing the position information
of the area to be refreshed. In this demo, you can use it for creating TFT d
isplay window.
    color_p     : Color structure pointer, indicating the color data to be d
isplayed in the refresh area. In this demo, it reads the address as DMA inpu
t to transmit data to the SPI bus and completes the image drawing
```

- LVGL display callback function implementation
  - Demo location: **lib/LVGL.py**
  - Implementation method: In this example, to maximize the reduction of processor utilization, DMA is used for color data transmission. **color_p** is set as the read address, and the SPI bus output data register is set as the write address. The code is long and only shows part of it, please download the demo to view the full code.

```
def disp_drv_flush_cb(self,disp_drv,area,color_p):
    ......
    self.rgb565_swap_func(data_view, size)  # Convert color format

    self.LCD.setWindows(area.x1, area.y1, area.x2+1, area.y2+1) # Set LVGL i
nterface display position

    ...... # DMA configuration is executed immediately

    while self.dma.active(): # Wait for DMA to be idle
            pass

    self.disp_drv.flush_ready() # Notify LVGL that data transfer is complete
```

LVGL Widget Layout

In LVGL, we can create various user interfaces. The basic components of the interface are objects, also called widgets, such as buttons, labels, images, lists, charts, or text areas. In a interface, we can create multiple widgets simultaneously and set their positions, sizes, parent objects, styles, and event handlers and other basic properties.

- Create an LVGL interface object
  - Demo location: **examples/LVGL_example.py**
  - Function: Create an LVGL interface object and pass in LCD object as parameter

```
# Init WIDGETS
WIDGETS(LCD=LCD)
```

- LVGL create tile
  - Demo location: **examples/LVGL_example.py**
  - Implementation function: A Tile view is a container object whose elements, called tiles, can be arranged in a grid format. Users can navigate between tiles by swiping. Use the Tile view object to call add_tile(tileview, row_id, col_id, dir) to create a new tile on the row_id row and col_id column. dir can be lv.DIR.LEFT/RIGHT/TOP/BOTTOM/HOR/VER/ALL or a value to move to the adjacent tile in the given direction by sliding.

```
//Create a tile at (0,0) with support for sliding down to (0,1)
self.tv = lv.tileview(self.scr)
self.tile1 = self.tv.add_tile(0, 0, lv.DIR.BOTTOM)
```

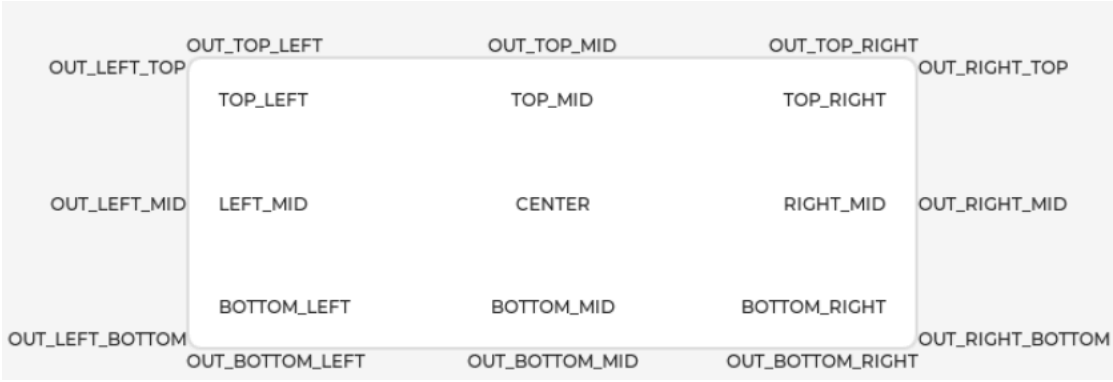
- LVGL create widget
  - Demo location: **examples/LVGL_example.py**
  - Implementation function: To create a widget, different widgets need to use different function interfaces, you can choose the parent object to create it.

```
//Create a table widget where tile2 is the parent object of the widget, whic
h can be replaced with a widget such as list, title, etc. that can have chil
d objects
self.table_imu_data = lv.table(self.tile2)
```

- Alignment positioning of LVGL widget
  - Demo location: **examples/LVGL_example.py**
  - Function: Enable a widget to be offset and positioned based on a reference point. The widget aligns to the center of the offset reference point widget.
  - Alignment standard: The LVGL library supports both internal and external alignments. By default, the upper-left corner is the origin, the leftward as the positive horizontal direction, and the downward as

the positive vertical direction.

```
//Position the widget 15 pixels to the right from the center point
self.table_imu_data.align(lv.ALIGN.CENTER, 15 ,0)
```



(/wiki/File:RP2040-Touch-LCD-1.28-LVGL-align.png)

- LVGL widget adjust font size
  - Demo location: **lv_micropython/lib/lv_bindings/lv_conf.h** and **examples/LVGL_example.py**
  - Implementation function: In actual use, an interface may need to use multiple font sizes. Multiple font sizes can be enabled in **lv_conf.h** and the default font size can be set. When setting the font size, it is necessary to stylize the widget so that it can be rendered according to the set style. The **add_style** function can be used to render various parts of the widget in different states.

```
#define LV_FONT_MONTSERRAT_16 1                                 // Enable fon
t 16
#define LV_FONT_MONTSERRAT_18 1                                 // Enable fon
t 18
#define LV_FONT_DEFAULT &lv_font_montserrat_18                  // Set the de
fault font size as 18

table_imu_data= lv.style_t()                              // Create an object
table_imu_data.init()                                     // Initialize
table_imu_data.set_text_font(lv.font_montserrat_16)       // Set the font size
as 16

self.table_imu_data.add_style(style_imu_table, 0)         // Set the style
```

Resources

Supporting Resources

Demo

- Demo (https://files.waveshare.com/wiki/RP2350-LCD-1.47/RP2350-LCD-1.47.zip)
- LVGL Demo (https://files.waveshare.com/wiki/RP2350-LCD-1.47/RP2350-LCD-1.47-LVGL.zip)

Schematic Diagram

- Schematic (https://files.waveshare.com/wiki/RP2350-LCD-1.47/RP2350-LCD-1.47-A.pdf)

Datasheets

- ST7789V3 (https://files.waveshare.com/wiki/common/ST7789V3_V0.1.pdf)

- 1.47inch LCD Datasheet (https://files.waveshare.com/wiki/RP2350-LCD-1.47/1.47inch_LCD_Datasheet.pdf)

Official Resources

Raspberry Pi Official Documents

- Get Started with MicroPython on Raspberry Pi Pico (https://hackspace.raspberrypi.org/books/micropython-pico)
- Raspberry Pi related books download (https://magpi.raspberrypi.org/books)
- Pico2 Schematic diagram (https://files.waveshare.com/wiki/common/Pico-2-schematic.pdf)
- Pico2 Pinout definition (https://files.waveshare.com/wiki/common/Pico-2-Pinout.pdf)
- Pico2 Getting Started (https://files.waveshare.com/wiki/common/Getting-started-with-pico-2.pdf)
- Pico2 C SDK User Manual (https://files.waveshare.com/wiki/common/Raspberry-pi-pico-2-c-sdk.pdf)
- Pico2 Python SDK User Manual (https://files.waveshare.com/wiki/common/Raspberry-pi-pico-2-python-sdk.pdf)
- Pico2 Datasheet (https://files.waveshare.com/wiki/common/Pico-2-datasheet.pdf)
- RP2350 Datasheet (https://files.waveshare.com/wiki/common/Rp2350-datasheet.pdf)
- RP2350 Hardware Design Reference Manual (https://files.waveshare.com/wiki/common/Hardware-design-with-rp2350.pdf)

Raspberry Pi Open Source Demos

- Raspberry Pi official C/C++ Demos (github) (https://github.com/raspberrypi/pico-examples/)
- Raspberry Pi official micropython Demos (github) (https://github.com/raspberrypi/pico-micropython-examples)

Development Software

- Thonny Python IDE (Windows version V3.3.3) (https://files.waveshare.com/wiki/common/Thonny-3.3.3.zip)
- Pico environment building related software (https://drive.google.com/file/d/110wdNeJrX-NhCtEoGBAZgOCin9_VkGH3/view?usp=sharing)
- pico-vscode package (https://drive.google.com/file/d/18-KDNrQlI0KuTMdS6W5iblUGaGm3FbVJ/view?usp=sharing)
- Zimo221 Chinese character conversion software (https://files.waveshare.com/wiki/common/Zimo221.7z)
- Image2Lcd image bitmap conversion software (https://files.waveshare.com/wiki/common/Image2Lcd2.9.zip)
- Font library conversion tutorial (https://www.waveshare.com/wiki/E-Paper_Font_Tutorial)
- Image bitmap conversion tutorial (https://www.waveshare.com/wiki/Image_extraction)

FAQ

Question: Raspberry Pi Pico 2 GPIO is configured as a pull-down input, why does reading IO show a high voltage level when the pin is left unconnected?

Answer:

You can refer to the RP2350-E9 section in the RP2350 Datasheet (https://files.waveshare.com/wiki/common/Rp2350-datasheet.pdf)

Support

Technical Support

If you need technical support or have any feedback/review, please click the **Submit Now** button to submit a ticket, Our support team will check and reply to you within 1 to 2 working days. Please be patient as we make every effort to help you to resolve the issue.
Working Time: 9 AM - 6 PM GMT+8 (Monday to Friday)

Submit Now (https://service.waveshare.com/)

Retrieved from "https://www.waveshare.com/w/index.php?title=RP2350-LCD-1.47-A&oldid=109203 (https://www.waveshare.com/w/index.php?title=RP2350-LCD-1.47-A&oldid=109203)"